CS310 Computer Science Project

Educational Software to train Air Traffic Controllers

Final Report

Supervisor: Professor Jane Sinclair

Author: Kazki Fairman | 2005639

Year of Study: 3rd Year MEng

May 02, 2023

Keywords: Simulator, Desktop App, Java, Aviation, Air Traffic Control, Aviation

Abstract

Air traffic controllers are subject to busy, highly stressful environments moving aircraft safely around the skies and airports. To achieve this with efficiency, controllers must thoroughly know strict procedures to ensure safety, as well as having excellent ability in spatial-awareness, problem-solving, and concentration. The project intends to build a graphical and interactive simulator to educate and train users to develop the necessary skills and knowledge in air traffic control. The simulator application will be made simple to entice people to try the simulator yet feature a moderate level of detail to enhance the experience of realism. The final application features a reliable simulator where the user can control aircraft at an airport, or within an airspace offering experiences of both perspectives of air traffic control. An educational aspect is provided through an informative tutorial as well as users learning from experience. Users are offered a simulator which can be used for leisure, or to experiment in one's competency of controlling air traffic.

Acknowledgements

I greatly appreciate all the guidance and supervision that Professor Jane Sinclair has provided me with during this project. I also wish to thank my fellow course mates Bailey Walters and Will Ly for allowing me to reflect on their opinions about my software when evaluating this project.

Contents

Abstract 2
Acknowledgements 3
Contents 4
List of Figures 7
List of Tables 10
1 Introduction 12
1.1 Existing Systems 12
1.2 Motivation 13
2 Project Requirements 15
2.1 Original Requirements 15
2.2 Requirements Evolution 19
2.3 Final Requirements 21
3 Project Management 24
3.1 Methodology 24
3.2 Original Project Plan 24
3.3 Changes to Project Plan 25
3.3.1 Problems and Setbacks 25
3.3.2 Decisive Project Management 26
3.4 Final Timeline 27
3.5 Ethical, Legal, Professional, and Social Issues 28
4 Software Design 29
4.1 Software Architecture 29
4.2 Tools Used 29
4.3 Object-Oriented Design 30
4.4 Application Design 32
5 Technical Implementation 35
5.1 Implementation of Requirements 35
5.1.1 Simulating the 3D space – (Requirement 1A) 35
5.1.2 Correctly representing planes on-screen – (Requirement 1B) 35
5.1.3 Detecting whether a violation of ATC rules have occurred (Requirement 1C) 38

5.1.4 Representation of non-aircraft entities (Requirement 1D) 41
5.1.5 Issuing and executing control commands for the aircraft from the user (Requirement 1E) 46
5.1.6 Tutorial Mode (Requirement 2A) 55
5.1.7 Normal Continuous Mode (Requirement 2B) 57
5.1.8 Realism (NFR-1) 57
5.1.9 Usability (NFR-2) 58
5.1.10 Performance (NFR-3) 60
5.2 Notable Technical Achievements 62
5.2.1 Graph Theoretic Structure of Airports 62
5.2.2 Modified A* Search for Realistic Maneuvering of Aircraft on Taxiways 63
5.2.3 Queueing of aircraft 64
5.2.4 Preventing deadlock traffic jams 65
5.2.5 Speed Recommendation System 66
5.2.6 A customized Mathematics library 68
5.3 Coding Practices 69
6 Testing 71
6.1 Functional Testing 71
6.1.1 Inspection Testing of Simulator Features 71
6.1.2 Unit Testing of Mathematical Functions 74
6.2 Remarks on Non-Functional Qualities 76
7 Evaluation 77
7.1 System Evaluation 77
7.1.1 Informal Opinions 77
7.2 Requirements Evaluation 78
7.3 Project Evaluation 79
8 Conclusions and Future Work 81
8.1 Project Outcome 81
8.2 Project Extensions 81
8.3 Author's Assessment 83
9 Glossary 84
10 References 85

11 Appendix 90
A – Structure of Simulator Components 90
B – UML Class Diagram (December 2022) 91
C – Table of Source File Purposes 93

List of Figures

Figure 1.1: Screenshot of Airport Madness 2 game by Big Fat Simulations Inc. 12
Figure 1.2: Screenshot of ATCPro simulator by Flag Mountain Software LLC and Aerius Designs. 13
Figure 1.3: Screenshot of Tower!3D Pro by FeelThere. 13
Figure 3.1: Gantt chart for original project plan. 25
Figure 3.2: Gantt chart for how the project unfolded. 27
Figure 4.1: Example of static polymorphism with the distance() method in MyMath.java. 30
Figure 4.2: Class diagram showing airplane instantiation using a factory design pattern. 31
Figure 4.3: Code snippet for loading and storing the airplane icon (in ImageLoader.java). 31
Figure 4.4: Design of main menu showing choice of environment (left), and choice of airports (right). 32
Figure 4.5: Screenshot of airport simulator. 33
Figure 4.6: Screenshot of airspace simulator, with Tokyo Haneda (HND) in the left, and Tokyo Narita (NRT) on the right. 33
Figure 4.7: Screenshot of airplane controls in airspace simulator (left), and airport simulator (right). 34
Figure 5.1: Code snippet of the Airplane class having a Point2D JavaFX object for its position, and a heading value (in AircraftController.java). 35
Figure 5.2: Aircraft representation, at the airport (left), in-flight within an airspace (right). 36
Figure 5.3: Calculations to find the change in coordinates of a plane for each 'time tick'. 36
Figure 5.4: Code snippet of movePosition in the Airplane class (in AircraftController.java). 37
Figure 5.5: Mini map showing airborne aircraft departing or arriving at airport (airport simulator). . 37
Figure 5.6: Equation to find lateral distance between two planes at two points. 38
Figure 5.7: A screenshot of the airspace simulator showing a dangerous minimum separation situation arranged deliberately. 38
Figure 5.8: Code snippet showing the LinkedList to store the *active occupant* aircraft, as a property of the Runway object (in EnvironmentController.java). 40
Figure 5.9: Plane (BA349) attempting to land on an occupied runway, leading to a go-around and message alert, before (top), and after (bottom). 40
Figure 5.10: Illustrated concept of the airport encoding in the text file, and a visual graph of the airport. 41
Figure 5.11: Flowchart to show the line-by-line airport construction logic for the method loadAirportFile() (in EnvironmentContoller.java). 43
Figure 5.12: Screenshot of airport simulator showing taxiways intersecting at a variety of angles. ... 45
Figure 5.13: Taxiway centerlines with constant radius at London Heathrow airport [25]. 45
Figure 5.14: Screenshot of AHS controls for aircraft in the airspace simulator. 46

Figure 5.15: Code snippet for the altitude control parameters (in AirborneAircraftControls.java). 47
Figure 5.16: Code snippet of Altitude class to handle the altitude of a plane (in AircraftValue.java). 47
Figure 5.17: Code snippet for code to check if the plane has met criteria to begin final approach and nominate an approach runway (in AirborneAircraftControls.java). 48
Figure 5.18: Screenshot of airspace simulator showing BA66 cleared for final approach to HND (Tokyo Haneda Airport). 48
Figure 5.19: Code snippet to check whether a plane has failed the approach attempt (in AirborneAircraftControls.java). 49
Figure 5.20: Flowchart to illustrate logic for planes following a path (for the airport simulator). 49
Figure 5.21: Visualized example of how the current target pointer and path stack works between passing a target node, before (top) and after (bottom). 50
Figure 5.22: Permission set for segments of two plane's paths crossing a runway at a single point and over multiple points before authorization (top), and after (bottom). 52
Figure 5.23: Screenshot of BA14 turning to the next waypoint on its path. 53
Figure 5.24: Screenshot of mini map with controls for selecting open runway approaches. 54
Figure 5.25: Screenshot of green runway exit selectors, with a yellow circle for the selected exit. 55
Figure 5.26: Highlighted parking gate for arrived aircraft to arrive at. 55
Figure 5.27: Screenshot of menu showing JavaFX CheckBox for tutorial mode selection. 56
Figure 5.28: Screenshot of airport simulator showing tutorial tooltips. 56
Figure 5.29: Code snippet showing tooltip cycling method changeTooltip() in TutorialManager.java. 57
Figure 5.30: Code snippet of message to strobe the tooltip opacity and change the tooltip every 8.5s (in TutorialManager.java). 57
Figure 5.31: Screenshot of the airport simulator. 59
Figure 5.32: Code snippet showing code in class AircraftTimer (in AircraftController.java) relevant to calculating and sending the correct elapsed time to commit to parts of the system each 'tick'. 61
Figure 5.33: Code snippet showing function moveFleet() (in class Fleet, in Aircraftcontroller.java), receiving and further propagating the elapsed time to commit to the simulation. 61
Figure 5.34: Code snippet showing function movePosition() in classclass Airplane, in Aircraftcontroller.java), receiving and putting the elapsed time into effect with a plane's position. 61
Figure 5.35: Optimal path found using traditional A* Search vs. modified A* Search. 63
Figure 5.36: Code snippet to show penalization of using a runway segment to route aircraft around the airport (in NodeHeuristic.java). 64
Figure 5.37: A diagram of when a plane may 'detect' another plane in front and queue behind it. 64
Figure 5.38: Code snippet to show the logic of the queuing system to avoid ground collisions. 65
Figure 5.39: Behavior of planes 'queueing' behind one another. 65
Figure 5.40: Diagram showing planes getting stuck at an intersection. 66

Figure 5.41: Flowchart to show process for when a lock is awarded to a taxiway and released. 66
Figure 5.42: Flowchart showing logic of an aircraft's speed decision process (for airport simulator). 67
Figure 5.43: Code snippet for resolveHorizontal and resolveVertical methods in MyMath.java. 68
Figure 5.44: Code snippet for rotateVector method in MyMath.java. 68

List of Tables

Table 2.1: MoSCoW analysis on requirements for development phase 1 (simulation engine). 17
Table 2.2: MoSCoW analysis on requirements for development phase 2 (modes of use). 18
Table 2.3: MoSCoW analysis on the refined requirements for development phase 1 (simulation engine). 23
Table 2.4: MoSCoW analysis on the refined requirements for development phase 2 (modes of use). 23
Table 2.5: MoSCoW analysis on the final non-functional requirements. 23
Table 5.1: Format for airport encoding. 42
Table 5.2: Format for airspace encoding. 44
Table 5.3: Description of classes used to represent airport entities. 44
Table 5.4: Description of classes used to represent airspace entities. 45
Table 6.1: Inspection tests of Simulator Features 73
Table 6.2: Discrete unit tests for MyMath functions. 76

This page is intentionally left blank.

1 Introduction

Air Traffic Control (ATC) is the process of monitoring and controlling airplane to move traffic efficiently, and safely. Training to become an Air Traffic Controller is a grueling and long process often taking around 3 years to complete all stages [1]. Either for leisure purposes, or to experiment one's competency in controlling air traffic, it can be helpful for there to exist simulators which allow users to control air traffic. These simulators may be as simple as a children's game, or as complicated as in reality.

1.1 Existing Systems

There are various ATC simulators for users, whether to learn about being an air traffic controller, or to test their spatial-awareness, problem solving, and concentration skills. Many of those simulators resemble a game designed for children, which limits how invested users can become due to its simplicity. One example is the game Airport Madness 2 shown in **Figure 1.1**. This simulator is a simple game which has a narrow strategic possibility; controls are limited to clearing planes for take-off, sending planes on a go-around, letting planes cross a runway, and taxing planes around the airfield with no routing control. Nevertheless, the friendly graphics and simplicity provides for good entertainment and is ideal for a young audience.

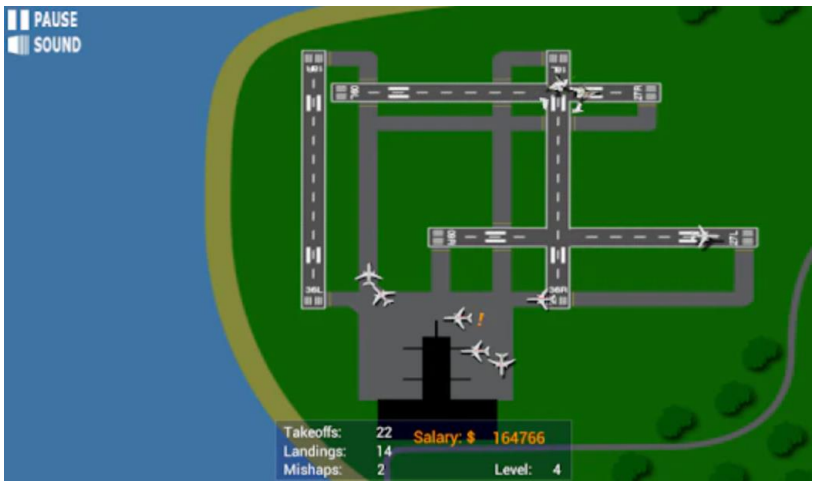


Figure 1.1: Screenshot of Airport Madness 2 game by Big Fat Simulations Inc.

On the other hand, some ATC simulators are extremely complex and true-to-life. These simulators pose a drawback in that they do not attract as many users because a higher level of commitment is necessary to such a complicated and professionally designed system. An example of such a simulator is ATCPro, shown in **Figure 1.2**. This is extremely realistic as it features voice input, levels of detail close to reality, and complex aspects of ATC. Such a simulator can be seen as one of the best in the market, as it has been designed by real Air Traffic Controllers and software experts [2].

Figure 1.2: Screenshot of ATCPro simulator by Flag Mountain Software LLC and Aerius Designs.

A final example of an ATC simulator is the Tower!3D Pro ATC simulator which is unique as it offers an interactive real-time 3D environment in which to control the aircraft. Along with this there are plenty of tools such as an airfield map and radar map visible in **Figure 1.3**. This is not as complicated as a simulator compared to ATCPro, however is more graphically intensive and costs $45 USD which can be a significant investment. Furthermore, a strong selling point is the ability to play multi-player, although this means one's friends need to also purchase this game to utilize this aspect [3]. Sadly, the freely available airports are limited, and extra airports must be purchased for approximately £15 GBP each [4], which can very quickly lead to a high expense on this single game.

Figure 1.3: Screenshot of Tower!3D Pro by FeelThere.

1.2 Motivation

Games for children such as Airport Madness 2 are enjoyable to use but do not offer a realistic experience for managing air traffic. Conversely, high complexity simulators such as ATCPro can be

cumbersome to learn as there are numerous specific rules to follow when controlling air traffic. No simulators have been found which offer an intermediate level of realism that has any sort of interactive learning, scenario customizability, and performance feedback. If such a simulator existed, users could become invested in controlling air traffic in an environment with moderate realism, without needing a high level of commitment. Such a 'gap-in-the-market' is the motivation to create an ATC simulator for this project; a simulator which has the right balance of accuracy and realism, versus simplicity.

Furthermore, a quality which all these examples lack is having both an airspace simulator and an airport simulator. To have a product which offers a complete experience of being an air traffic controller, ideally it should have both an airspace and airport simulator. Therefore, both will be featured in this project to ensure superiority in this aspect.

A more external motivator for this project is that there are reports of shortages in air traffic controllers in the UK [5], and it can be hoped that the simulator designed in this project will encourage people to consider ATC as a career option.

Finally, from a personal perspective, commercial aviation is of great interest. Therefore, it is suitable to develop a project linked to commercial aviation and specifically ATC as there is a passion for development.

2 Project Requirements

2.1 Original Requirements

Since the initial project objectives are vague in description, it was beneficial to derive a set of project requirements from the project which are much more specific in what they include (initial objectives and requirements in **Table 2.1** and **Table 2.2**). For example, objective 1e only gives an idea about using some GUI to provide commands to aircraft without detail, and this has been improved with the corresponding requirements 1e-1 through 1e-3. The priority of each individual requirement will be determined through a MoSCoW analysis. This would provide a sensible guideline for what should be implemented during the development of the project should time available or other factors constrain the project scope. A requirement is prioritized as:

- **Must Have** if project success is dependent on this requirement,
- **Should Have** if it's high-priority and should be implemented, if possible,
- **Could Have** if its desirable but not necessary, or
- **Won't Have** if it will not be implemented but may be considered in the future [6].

A disadvantage of these original requirements is that even though they are an improvement compared to the initial objectives, they lack specificity and detail in how the requirements will be implemented into the product. This absence of detail was due to the preliminary nature of these requirements which would later become a more refined set of requirements as details and possibilities become known. Although quite difficult to achieve, ideally a project should have all its objectives be SMART: specific, measurable, achievable, realistic, and time-bound [7]. The final requirements (see **2.3 Final Requirements**) are more detailed and specific.

Initial Objective	Req- ID	Requirement	Priority
Objective 1a: Simulate a 3D space, represented as a 2D radar interface to the user.	1a-1	Simulate aircraft in the lateral dimensions using a pair of numbers (coordinates).	MUST
	1a-2	Simulate aircraft in the vertical dimension using a single number (altitude).	MUST
Objective 1b: Represent aircraft on-screen with data pertaining to each aircraft, such as altitude and speed.	1b-1	Display each aircraft on-screen in the correct position according to its coordinates in the lateral dimensions	MUST
	1b-2	Display the direction of each aircraft.	MUST
	1b-3	Display the live altitude of each aircraft	MUST
	1b-4	Display the live speed of each aircraft	MUST
	1b-5	Display the destination (where applicable) of each aircraft.	MUST
Objective 1c: Verify whether ATC regulations are being violated or not,	1c-1	Detect and notify when minimum separation (laterally and vertically) between aircraft is violated.	SHOULD (given 1a-1 & 1a-2)

(e.g., maintaining minimum separation between aircraft).	1c-2	Detect and notify if a runway is used in a *dangerous* manner. **Remark:** Using a runway *dangerously* refers to events where there exists a risk of collision (e.g., two planes landing simultaneously).	SHOULD (given 1d-2)
	1c-3	Detect and notify when a plane enters a no-fly zone.	MUST (given 1d-7)
	1c-4	Detect and notify when distance is not left between planes considering wake turbulence.	SHOULD
Objective 1d: Represent other objects/entities, such as airports, taxiways, waypoints, restricted zones.	1d-1	Represent and display taxiways which can be used by aircraft on the ground to taxi.	MUST
	1d-2	Represent and display runways which can be used by aircraft on the ground to taxi, takeoff, and land.	MUST
	1d-3	Represent and display gates which can be used by aircraft on the ground.	MUST
	1d-4	Represent and display an airport using taxiways, runways, and gates for aircraft on the ground.	MUST
	1d-5	Represent and display waypoints in the sky for aircraft which are airborne.	MUST
	1d-6	Represent and display an airport as a set of runways for aircraft which are airborne.	MUST
	1d-7	Represent and display no-fly zones for aircraft which are airborne.	SHOULD
Objective 1e: Present a GUI to the user which can accept input control commands for the aircraft.	1e-1	Allow a user to click on the visual representation of an aircraft on-screen to select an aircraft (in preparation to control the aircraft).	MUST
	1e-2	Allow the user to click a button to issue a command to a selected aircraft at an airport for: taxiing, takeoff clearance, landing clearance, and go-around.	MUST
	1e-3	Allow the user to click a button to issue a command to a selected aircraft within an airspace for: approach clearance, change of altitude, change of heading, and change of velocity.	MUST
Objective 1f: Simulate special events, such as adverse weather or emergencies.	1f-1	Simulate maintenance works on an airport; a closure of a set of taxiways and/or runways.	COULD
	1f-2	Simulate heavy rainfall and the effect on aircraft (less braking action for landing aircraft).	COULD
	1f-3	Simulate crosswinds of varying intensity and the effect on aircraft (more pilot initiated go-arounds).	SHOULD
	1f-4	Simulate pilot deviation (aircraft making an uncommanded move).	WONT
	1f-4	Indicate if an aircraft is experiencing an emergency.	SHOULD

Objective 1g: simulate important elements of realism, such as pilot delay between being issued a command from ATC and the aircraft reacting to the command.	1g-1	Simulate delay between issuing of commands from the user and the aircraft performing that command.	SHOULD
	1g-2	Simulate the effects of true airspeed compared to indicated airspeed for aircraft at different altitudes (this is a result of air pressure differences across altitudes resulting in uncalibrated readings of an aircrafts speed) [8].	COULD
	1g-3	Simulate the different flying characteristics of different types or categories of aircraft.	SHOULD
Objective 1h: Provide performance metrics as feedback to the user on their competency.	1h-1	Calculate and display a metric for the user's efficiency.	SHOULD
	1h-2	Calculate and display a metric for the user's safety / adherence to ATC regulations.	SHOULD
Objective 1i: Pause, save, and load runs/scenarios.	1i-1	(Let the user) save the state of a simulation.	SHOULD
	1i-2	(Let the user) open the state of a previously saved simulation.	SHOULD
	1i-3	(Let the user) pause a simulation.	SHOULD
Objective 1j: Provide dynamic warnings or hints to the user based on the live simulation.	1j-1	Suggest to the user to instruct a pending command to a plane (e.g., suggest to takeoff if the plane has been idling on the runway).	COULD
	1j-2	Suggest to the user to take issue a diversionary vector to an aircraft if it may fly to danger (close to another plane or a no-fly zone).	COULD

Table 2.1: MoSCoW analysis on requirements for development phase 1 (simulation engine).

	Req- ID	**Requirement**	**Priority**
Objective 2a: Educational modes, which interactively teaches the user on the objectives and rules of controlling air traffic.	2a-1	Advise the user how to perform the issuing of commands within the simulator.	SHOULD
	2a-2	Verify whether the user has performed the advised command (to progress the state of the tutorial).	MUST (given 2a-1)
Objective 2b: Practice modes, which allow the user to practice specific scenarios (e.g., controlling airport traffic with a closed runway).	2b-1	Start the simulator on an airport with a set of taxiways and/or runways out of action.	COULD
	2b-2	Start the simulator on an airport with a specified weather setting.	SHOULD
	2b-3	Start the simulator on an airport with a specified traffic rate.	SHOULD
Objective 2c: Survival mode, where the user must control airport/airspace traffic for as long as	2c-1	Run the simulator with an increasing traffic rate over time.	COULD
	2c-2	Count how many violations of ATC regulations take place.	MUST (given 2c-1)

possible without incurring an infringement of the regulations, testing the user's competency and consistency.	2c-3	Stop the simulation if too many violations of ATC regulations take place.	MUST (given 2c-1)
Objective 2d: Continuous mode, which is a 'free-play' run on a customizable scenario for the user to use however they please.	2d-1	Start the simulator with no special settings.	MUST

Table 2.2: MoSCoW analysis on requirements for development phase 2 (modes of use).

An important note is that objectives 1a through to 1e were considered as the core functionality for the simulation engine; without these features the simulator would be undoubtedly insufficient. For example, without the ability to represent an aircraft in a 3-D space (vertical dimension; altitude, and lateral dimensions), the simulator would be an idle empty box. If aircraft did not have the ability to use a taxiway or to takeoff, then the most basic maneuvers of an aircraft at an airport would be impossible to simulate. Hence, nearly all requirements derived from objectives 1a through to 1e are prioritized as MUST.

On the other hand, objectives 1f through to 1j were not considered as critically necessary for the simulation engine but were considered as important for the final product to provide the intended experience to the user. For example, the simulator being able to simulate weather events would let the user practice their air traffic controlling abilities in abnormal situations. The feature of providing a performance metric to the user based on their live performance is not critically necessary but can be beneficial as the feedback to the user may help them improve their air traffic controlling skills. Therefore, while none of the derived requirements 1f through 1j are prioritized as MUST, many are prioritized as SHOULD or COULD.

Some further objectives were also written in the **Project Specification** as potential extensions to the project if time allowed:

> **Objective 3**: A graphical airport/airspace builder, which offers users the chance to make their own environment to test their ATC skills in. This also allows the chance to generate an environment based on real airports/airspaces in the world.
>
> **Objective 4**: Using automated voice callouts from aircraft, which provides an additional level of realism, input affirmation, and convenience to the user. E.g., the controller (user) hearing pilots (non-user) acknowledge "Cleared to land RWY 27L".

Objective 5: Replay feature which allows users to replay their run on the simulation, at a normal or modified speed. This may be useful for users to review their actions in a past run.

As interesting as some of these extra objectives seem, sadly none of these extra objectives were implemented due to the time constraints during the project. Since these objectives were only ever considered as exploratory ideas for project extensions, there was no derivation of requirements or a MoSCoW analysis on objectives 3, 4, or 5.

2.2 Requirements Evolution

During development, unexpected issues arose meaning the project could not progress as planned (discussed in **3 Project Management**) and therefore, some of the requirements had to be modified or removed. The advantage of using a methodology which is not strictly plan-based is that requirements can be re-visited and amended throughout the project. To a large extent, the changes in the requirements were a reduction in scope due to limited time. Although there will be some subjectivity about which requirements are best to modify or remove, decisions were made diligently with respect to the project's foreseeable future. This section discusses the decision making involved with the changes of the original requirements.

Correctly representing airplanes on-screen (Objective 1b)

When using the simulator for controlling traffic at an airport, the airport will take up the majority of the space on the screen. While this is useful to be able to interact with various parts of the airport on the system, it causes a problem of planes not being visible when they are approaching the airport to land, especially when they may be a few nautical miles away from the runway. To solve this problem, a new requirement was generated to implement a small mini-map when controlling traffic at an airport, to show planes that are on their way to land at the airport. This new requirement (1b-6 in **Table 3.3**) will help the user expect when a plane may appear on their main screen that is landing. This does not affect the airspace simulator, only the airport simulator.

Detecting violations of ATC regulations (Objective 1c)

This objective is important as without this, there would be no detection of dangerous situations which may occur when a user is using the simulator. However, due to the time constraints of the project, a difficult decision to remove some scope here was taken. Therefore, requirements 1c-3, and 1c-4 were completely removed, and 1c-2 was reduced to a requirement more limited in scope. Due to the aforementioned importance of this project objective, the requirements deemed to be most important (1c-1 and 1c-2) were not removed from the project – it is especially important to detect whether two planes are flying too close to each other. On the other hand, the removed (1c-3 and 1c-4) requirements were deemed to be less important, and slightly more complex to implement.

Representing non-aircraft entities (Objective 1d)

This objective did not experience any large additions or removals but was modified to become much more specific. At an early stage of the project, it became obvious that a graph theoretic framework would be the most advantageous method (see **5.2.1**) to implement the network of infrastructural entities (runways, taxiways, and parking gates) of an airport. Therefore, a sensible decision was made to incorporate an additional set of requirements within this objective to set out the necessities of this graph theoretic implementation. Furthermore, the existing requirements were re-worded to improve their specificity. For example, requirement 1d-1: *"Represent and display taxiways which can be used by aircraft on the ground to taxi*." leaves some crucial details unanswered – How will the aircraft navigate around the taxiway network? How will the aircraft handle the boundary between a taxiway and a runway? Such gaps in the design of the planned simulator could have led to a standstill if solutions were not found to such problems. Therefore, many missing details such as these are cleared up in the re-written final requirements.

Additionally, it was decided that no-fly zones (requirement 1d-7) would not be included in the final product. This sacrifice was another reason for requirement 1c-3 being made redundant.

Simulation of weather and events (Objective 1f)

Unfortunately, in a reconsideration of scope for the project, the requirements associated with this objective were removed. This was in the interest of completing features deemed more important.

Simulating elements of realism (Objective 1g)

Similar to objective 1f, this was removed for the same reason. For example, it would be nice to have the simulator include different types of planes with different flying characteristics (e.g., typical landing speeds), but this is not deemed necessary, and the simulator will still be useful with aircraft being homogenous. However, as having a mediocre level of realism was envisioned to be a selling point of this project, a new set of requirements (NFR-1) were made to encourage the development to incorporate realism where possible (e.g., velocity of aircraft to scale with the map).

Performance Metrics (Objective 1h), Pause/Save/Loading runs (Objective 1i), Dynamic warnings/hints (Objective 1j)

The final three objectives for the simulation engine were sadly also removed due to time constraints and the anticipated complexity to implement the requirements. On the topic of providing an indicator of performance to the user – one's performance within the simulator should be clear with how well they are clearing traffic, or how many mistakes they realize they are making. By keeping the earlier mentioned requirements 1c-1 and 1c-2, notifications of violating ATC rules will still provide for some feedback of whether the user does some things wrong in the simulator or not.

Modes of use (Objective 2)

Again, due to the time constraints of the project, some of the scope was cut from these requirements. Objective 2b became redundant due to the removal of special events (weather, airport disruption etc.) from the simulator. Since it is important to have some sort of tutorial for the users to know how to use the simulator, the educational mode was kept. The continuous mode was also kept as this does not require any heavy implementation.

Non-Functional Requirements

Although it is useful to have requirements which stipulate the required functionality of the system, the original requirements lacked non-functional requirements which describe qualities for the system. For example, at no point did the original set of requirements specify that the system must run smoothly or have a clear GUI. Therefore, three non-functional requirements generated are centered around the simulator being realistic, usable, and well performing, shown in **Table 2.5**.

The non-functional requirement of making the simulator realistic was known to include many separate aspects such as the following.

- **Maps and movement to-scale.** This is a good realism quality to include as the simulator will have an experience as similar as possible to reality in terms of the size, speed, and scale of the physical entities.
- **Realistic flying characteristics.** It would be unhelpful to have a simulator where planes fly with unrealistic characteristics, such as performing a turn or an altitude change instantaneously, since this would ruin the challenge of skillfully maneuvering planes as it would be much easier. In real life, planes take time to climb, to turn, and to accelerate due to the physical limitations they experience.
- **Proper phraseology.** One may think that pilots and ATC communicate as regular speakers do in their own language, and simply ask “Can I takeoff now?”. However, this is far from the truth. Even in Hollywood movies there are inaccuracies, demonstrated by the film “2:22”, where instead of a controller instructing a plane to increase their speed according to proper phraseology, instructed the plane to “Punch it!” [9] [10]. This demonstrates a serious misconception about ATC communication and therefore this simulator must be careful not to encourage this incorrect habit, by ensuring correct terminology is used. The reason why using proper terminology is important in ATC is to promote universal understanding [11] which improves communication, therefore enhancing safety and efficiency.

2.3 Final Requirements

The requirements of the final product are shown in **Table 2.3**, **Table 2.4**, and **Table 2.5**. These requirements are formed from adaptations and amendments made during various stages of the project, some of which are discussed in **2.2 Requirements Evolution**.

Requirement Set	Req- ID	Requirement	Priority
(1A) Simulating the 3D space	1a-1	Simulate aircraft in the lateral dimensions using a pair of numbers (coordinates).	MUST
	1a-2	Simulate aircraft in the vertical dimension using a single number (altitude).	MUST
(1B) Correctly representing planes on-screen	1b-1	Display each aircraft on-screen in the correct position according to its coordinates in the lateral dimensions.	MUST
	1b-2	Simulate and display the direction of each aircraft.	MUST
	1b-3	Simulate and display the live altitude of each aircraft.	MUST
	1b-4	Simulate and display the live speed of each aircraft.	MUST
	1b-5	Display the destination (where applicable) of each aircraft.	MUST
	1b-6	Display airborne aircraft when using the airport simulator in an on-screen mini-map according to its coordinates in the lateral dimensions.	SHOULD
(1C) Detecting whether a violation in ATC rules have occurred.	1c-1	Detect and notify when minimum separation (laterally 3 nautically miles and vertically 1,000 feet) between aircraft is violated.	SHOULD (given 1a-1 & 1a-2)
	1c-2	Detect and notify if a plane is about to land on an occupied runway.	SHOULD (given 1d-2)
(1D) Representation of non-aircraft entities.	1d-1	Read the contents of a file which specifies the airport's taxiways, runways, and parking gates.	MUST
	1d-2	Construct a network of runways and taxiways using a graph theoretic structure.	MUST
	1d-3	Incorporate parking gates into the graph theoretic structure representing an airport	MUST
	1d-4	Represent and display taxiways which can be used by aircraft on the ground to taxi.	MUST
	1d-5	Represent and display runways which can be used by aircraft on the ground to taxi, takeoff, and land.	MUST
	1d-6	Represent and display gates which can be used by aircraft on the ground.	MUST
	1d-7	Allow aircraft to correctly interact with the infrastructure entities (e.g., navigate around the airport, wait before entering a runway).	MUST
	1d-8	Read the contents of a file which specifies an airspace, consisting of runways of airports and waypoints.	MUST

	1d-9	Represent and display waypoints in the sky for aircraft which are airborne.	MUST
	1d-10	Represent and display an airport as a set of runways for aircraft which are airborne.	MUST
	1d-11	Allow aircraft to correctly interact with the airspace entities (e.g., intercept the approach path to arrive at an airport, be able to follow a series of waypoints).	MUST
(1E) Issuing and executing control commands for the aircraft from the user.	1e-1	Allow a user to click on the visual representation of an aircraft on-screen to select an aircraft (in preparation to control the aircraft).	MUST
	1e-2	Allow the user to click a button to issue and execute a command to a selected aircraft at an airport for: clearing the taxi path, extending the taxi path, crossing a runway, takeoff clearance, permission to lineup and wait (on a runway), landing clearance, and go-around.	MUST
	1e-3	Allow the user to click a button to issue and execute a command to a selected aircraft within an airspace for: approach clearance, change of altitude, change of heading, change of velocity, to extend a path of waypoints to follow, to clear the path of waypoints to follow.	MUST

Table 2.3: MoSCoW analysis on the refined requirements for development phase 1 (simulation engine).

Requirement Set	Req- ID	Requirement	Priority
(2A) Tutorial Mode	2a-1	Advise the user how to perform the issuing of commands within the simulator, such as instructing a plane to turn to heading 240.	SHOULD
	2a-2	Have an interactive tutorial such as verifying whether the user has performed the advised command (to progress the state of the tutorial).	SHOULD
(2D) Normal Continuous Mode	2d-1	Start the simulator with no special settings.	MUST

Table 2.4: MoSCoW analysis on the refined requirements for development phase 2 (modes of use).

Req- ID	Requirement	Priority
NFR-1 – Realism	The simulator should represent realism wherever possible. Examples include but are not limited to: realistic flying characteristics (e.g., acceleration), realistic airports, realistic behavior of planes on ground, etc.	SHOULD
NFR-2 – Usable GUI	The simulator should have a usable GUI which is not overly confusing.	SHOULD
NFR-3 – Performance	The simulator must not suffer from extreme lag or unresponsiveness.	MUST

Table 2.5: MoSCoW analysis on the final non-functional requirements.

3 Project Management

3.1 Methodology

Due to the short time available to have developed the project and the expected likelihood of plans needing to change during development, a mix of plan-driven and agile practices were employed for the software development process. As in plan-driven projects, most testing was done at the end, and a plan was followed to develop the product; new scope ideas did not intermittently arise during the development. Additionally, as in agile projects, a short implementation cycle was carried out for each feature which facilitates an incremental development; each new feature could be considered a new version of the product. Each implementation cycle consisted of developing one or a few related features (e.g., the ability for airplanes to takeoff from a runway), then performing some basic testing to verify functionality. This mid-development testing was not conclusive however, as the final testing (**6.1**) was done at the end of the project to properly verify functionality, quality, and integrity of the system. In general, this was a good methodology to follow as it is possible to backtrack to a previous version of the project if there has been a serious mistake, or if feedback needs to be acquired before the full completion [12]. Additionally, if development was to take longer than expected, it would have been possible to release what has been built when time runs out [13].

On a separate note, backups will be regularly made to prevent a loss in progress. Regular backups will complement the incremental development strategy as once a new feature is implemented, this can be backed up and permanently saved as a new version of the software.

3.2 Original Project Plan

The original project plan (**Figure 3.1**) was based on the original project objectives as identified in the **Project Specification**. In this original project plan, there were two distinct development phases scheduled in sequence of one another. Phase 1 focused on the planned development of the simulation engine which would form the core functionality of the software. Whereas phase 2, which could not be started until Phase 1 was completed, focused on developing the modes of use for the ATC simulator; to use the simulation engine in different settings and parameters to provide various experiences to the user (e.g., an easy educational mode). The time between phase 1 and phase 2 can be considered as the half-way point of developing the whole application. Development phases 1 and 2 were scheduled to be mostly contained within term 1 and term 2 of the academic year respectively, with the winter break and spring break acting as a safety net in case of slow progress.

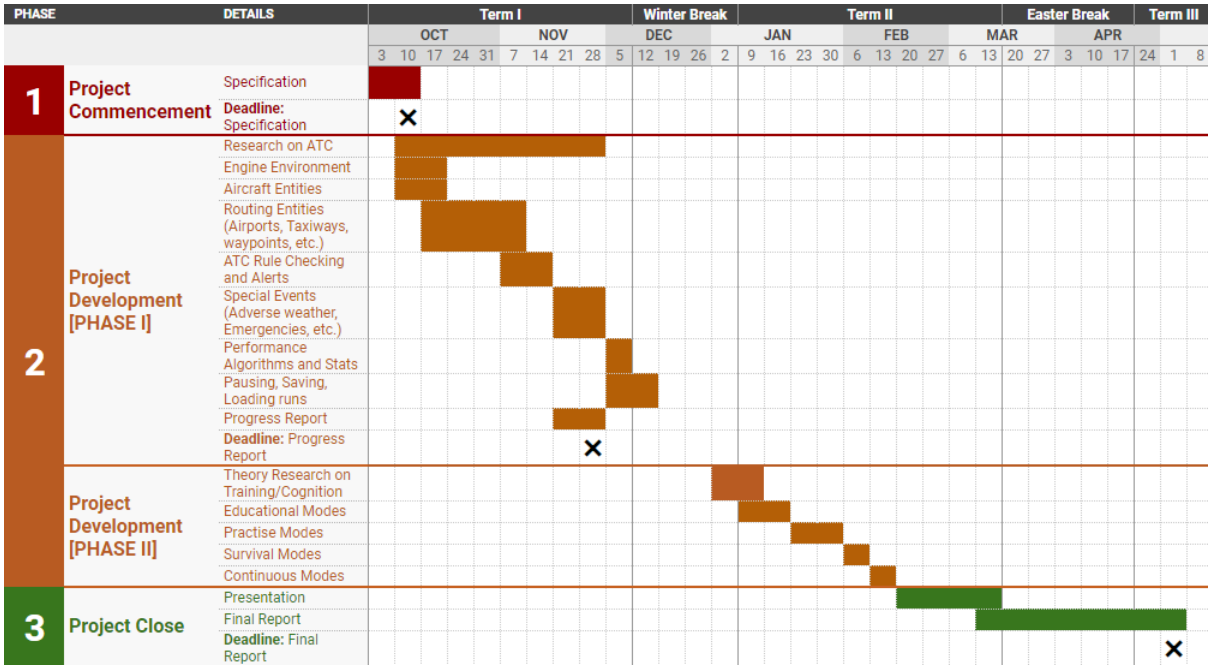


Figure 3.1: Gantt chart for original project plan.

Finally, the completion of the second development phase was expected by the end of the second term, leaving an abundance of time to focus on the project presentation and the project final report.

The original plan envisioned approximately, but not constraints to 10 to 20 hours dedicated to the project, depending on progress and other commitments. In addition, a weekly 15-minute meeting was planned with the supervisor to review progress, and seek any advice needed on the direction of the project.

3.3 Changes to Project Plan

As with any project, it is impossible to know exactly how events will unfold or how work progresses. In this project there were a handful of unexpected events which contributed towards the removal or modification of parts of the plan.

3.3.1 Problems and Setbacks

Originally, the a priori assumption was that progress on the implementation would be proportional to the quantity of time spent working on the project. However, as the solution became larger and more complex, progress slowed down due to taking time on the following.

- To avoid a disorderly implementation; implementing new features in a specific way which complies well with the existing implementation – a ‘threading the needle’ challenge.
- Refactoring the solution; re-structuring code and data structures to reduce complexity and increase versatility of the solution.

Time was taken to do this in the interests of product design and solution quality. The impact of these obstacles in development was foreseen but could not have been predicted to have been as hindering as they have been.

At the start of the project, a minor issue was not knowing how to utilize the JavaFX library. This was very quickly overcome by studying the convenient JavaFX documentation and following online tutorials.

One problem which consistently caused setbacks in the rate of progress was competing priorities. Some of the competing priorities culprit to slowing progress were other modules, applying for jobs, and dealing with the landlord. Whilst there was an effort to maintain a balance of time dedicated to all important matters, occasionally these competing priorities required more time than anticipated and therefore unfortunately started to reduce the time available to work on the project.

A final problem that was dealt with during the development of the project was the lengthy process of fixing bugs and errors in the solution. Spending time to fix this was unavoidable as it is not acceptable to leave bugs in the solution, however it took significantly longer than anticipated to do this, partly due to the aforementioned 'threading the needle' problem; adding new features sometimes conflicted with existing code. Although this was also partly due to my lack of experience programming; my experience is currently only a few years.

Combining all these problems and factors together, the schedule inevitably started to fall behind, which required some decisions to be made to prevent the project from being incomplete.

3.3.2 Decisive Project Management

Given the schedule had started to fall behind due to the problems and setbacks experienced, it became increasingly necessary to exercise some effective project management to avoid ending the project in an incomplete state. One significant decision was to reconsider the scope of the project and remove things that were not essential – this is extensively documented in **2.2 Requirements Evolution**. Effectively this made the project smaller and allowed more time to complete the more essential components of the simulator.

With the extent of underestimating the time needed to develop the core parts of the simulator, along with some ambitious claims about the remainder of the project in the **Progress Report**, it was also useful to spend time during the winter break, and spring break to further progress with the project. Fortunately, this eased the pressure of completing the project and this can be attributed to the original schedule (**Figure 3.1**) including the winter and spring breaks as a buffer zone for when time became of the essence – a contingency.

Despite the large reduction in scope making the project seem like it was not going to plan, it can be more suitable to be over-ambitious from the start and eventually reducing the scope, instead of being under-ambitious from the start and trying to expand the scope to fill the time. Although there could be some subjective disagreement on this, the latter may be a more dangerous approach because trying to expand a project plan to fill up some remaining time could either cause the project to move away from its original intention, or create obstacles if the project cannot be expanded in a suitable manner.

3.4 Final Timeline

We can observe that in the final timeline (**Figure 3.2**) that the number of components in the project has been reduced, due to Routing Entities being very complex and time-consuming by stretching a very long period. As mentioned in **3.3**, the scale and complexity of this component was severely underestimated. This timeline contains no changes regarding the submittable documents for the project (the specification, progress report, and final report) as these are mandatory to complete no matter how the project progresses.

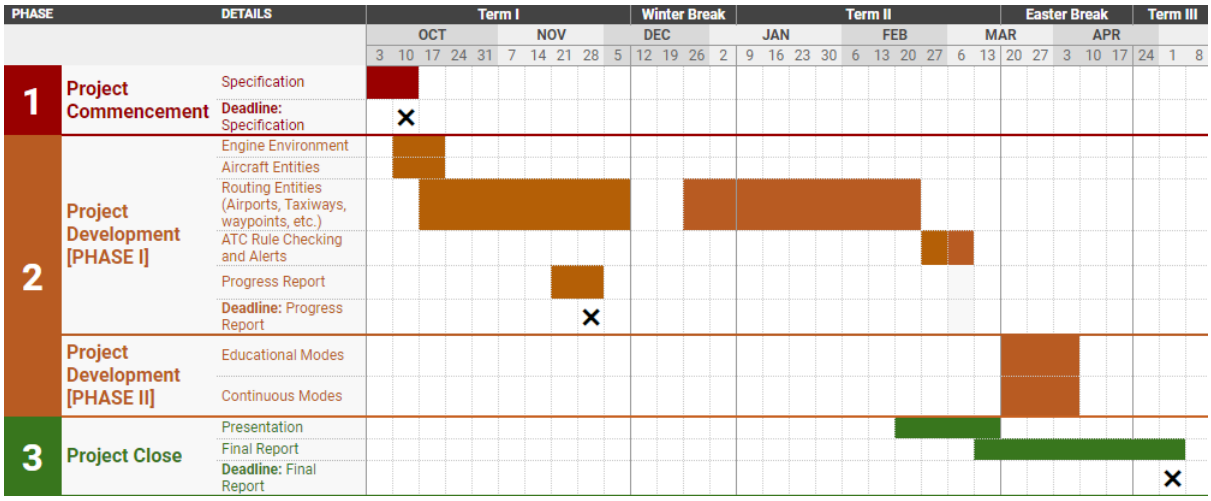


Figure 3.2: Gantt chart for how the project unfolded.

Although not labelled on the final timeline (**Figure 3.2**), there was a 15-minute weekly meeting with the supervisor which was helpful particularly when focusing on details of the project, and when needing to change the project plan. For this reason, the meetings can be considered successful, but also because both the supervisor and I attended on-time and were rarely cancelled.

Another detail which is missing from the final timeline (**Figure 3.2**) is the time spent on the project. As mentioned in **3.2**, the original intention was to have 10 to 20 hours each week during term-time on the project. In reality there ended up being sometimes less than 10 hours during a week on the project, but many weeks more than 20 hours due to the need to make progress, but also due to a personal sense of enjoyment to complete a project related to my interest in aviation. Overall, there ended up being more time working on the project than originally expected. This is a good indication that reasonable

time and effort was spent on the project, as opposed to not spending enough time and that being the reason for cutting scope or leaving parts incomplete.

3.5 Ethical, Legal, Professional, and Social Issues

Some basic opinions were informally gathered from close friends on their general thoughts as discussed in **7.1.1**. This was within the limits of an undergraduate project and does not constitute a data collection mission in its own right. In general, the project has not required any work with other people or the environment, and therefore no further ethical, legal, professional, or social considerations have arisen since producing the **Progress Report**.

4 Software Design

This chapter will focus on some of the design choices taken in the project from perspectives including architectural, developmental, and application design.

4.1 Software Architecture

One important note is that the UI implementation is closely intertwined with the functionality of the system; there is no isolated 'frontend' and 'backend' of the system. This is due to the nature of the system being written as a single Java application as opposed to having multiple system components (e.g., a python backend connected to a JavaScript frontend via an API). Despite this, in the interest of good coding practice, where possible, code responsible for displaying the GUI has been separated from the main functionality into their own classes and functions (see more in **5.3 Coding Practices**).

Since many aspects of the simulator (**Appendix**: **A – Structure of Simulator Components**) must work cohesively together (e.g., the displaying of aircraft and calculation of aircraft's position) to achieve an accurate simulation in real-time, it makes less sense to use a software architecture which is disjoint in nature, such as the Model-View-Controller architecture which uses three main logical components [14], or a multitier architecture. Therefore, a monolithic architecture has been used as this allows all parts of the system to be closely linked and available to each other, hopefully eliminating problems of latency or limited bandwidth between disjoint software components as in other architectures. It is possible that another architecture is more suited to the project, however choosing a fancy architecture which I am less familiar with may have hindered the rate of development.

A full class diagram of the system was considered, however was not produced as the diagram would have been too large and time-consuming; the class diagram at the time of the **Progress Report** (**Appendix**: **B – UML Class Diagram (December 2022)**) shows just how large and complicated the system was not even half-way through the project. A different diagram showing all the dependencies between components of the system is also pointless as this would be close to a complete graph; many components rely on most other components. Finally, to breakdown the purpose of different parts of the system, all files have their responsibility denoted in a table (**Appendix**: **C – Table of Source File Purposes**).

4.2 Tools Used

Given the simulator needs to handle interactions between various entities (e.g., a plane and a taxiway), this naturally lends to an object-oriented problem to implement, and therefore an object-oriented language was the best choice to use. The chosen object-oriented programming language was Java because of my familiarity with the language and its convenient JavaFX library to represent the visual

elements of the software. Alternative candidate languages were considered, but were not selected for the following reasons:

- The Python programming language supports object-oriented programming and is convenient to use, however was not selected due to having less experience in Python compared to Java.
- The C programming language was considered but was not selected as it is not object-oriented, and therefore would not be suitable to implement this solution.
- The language Haskell was considered but was quickly disregarded due to its functional programming nature being inappropriate for this sort of problem.

As much as choosing a suitable programming language is important, is it also important to consider a suitable software to help develop the solution to this project; an Integrated Development Environment (IDE). IDEs are very helpful and can be considered essential when designing software in the present day [15]. Although it is possible to use other software such as notepad or Microsoft Word, they severely lack the beneficial features to support development of code and software. Therefore, a very customizable and popular IDE, Visual Studio Code was selected and used throughout the entirety of the project to develop the solution.

4.3 Object-Oriented Design

The system heavily features object-oriented programming due to the various entities that must interact with each other. It is helpful to consider the advantages of such a design and of the various design patterns which have been employed.

Firstly, object-oriented programming offers benefits in that the code becomes modular and reusable [16] because of how code is written within classes and functions which can be instantiated and called. This in turn improves productivity as these advantages make it easier to develop the application. Additionally, polymorphism, which allows something to occur in several different forms, offers flexibility and eases development since the code can be more versatile. Both dynamic polymorphism (overriding methods using inheriting subclasses), and static polymorphism (overloading methods with differing parameter sets) [17] have been used to increase the flexibility of parts of the solution, with an example in **Figure 4.1**.

```
protected static double distance(Point2D source, Point2D destination) {
    return (Math.hypot(Math.abs(source.getX() - destination.getX()),
                       Math.abs(source.getY() - destination.getY())));
}

protected static double distance(EnvironmentController.NavigationNode source,
                                 EnvironmentController.NavigationNode destination) {
    return distance(source.coord, destination.coord);
}
```

Figure 4.1: Example of static polymorphism with the distance() method in MyMath.java.

Finally, some implementation patterns have been used to help the object-oriented design.

- **Factory Design Pattern**. The factory design pattern is when an abstract class or interface is defined for creating an object but lets the subclasses be responsible to create the instances [18]. This has been implemented where the abstract Airplane class is inherited and instantiated by the GroundAirplane and AirborneAirplane class (in AircraftController.java). Then, the Fleet class can be considered as a factory to decide which subclass to instantiate, whenever asked to by the AircraftTimer class (**Figure 4.2**). The benefit of this is that other parts of the system can interact with the abstract Airplane class in a general manner but can still function in the specific cases of ground airplanes (at an airport) or airborne airplanes (in an airspace).

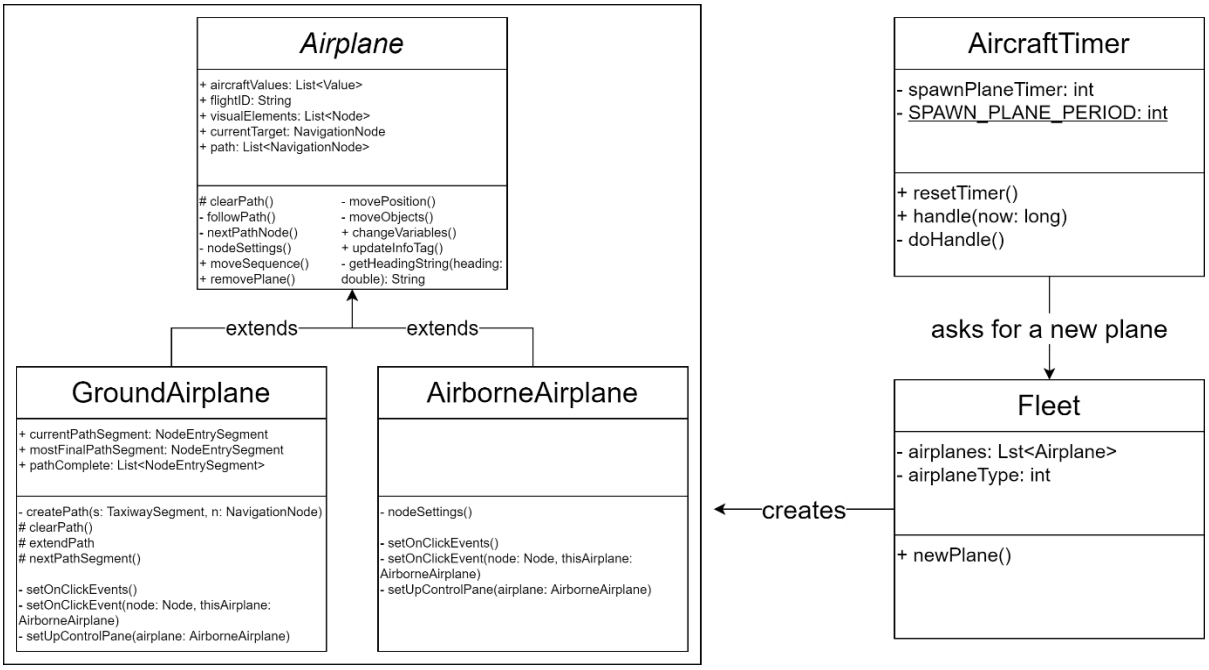


Figure 4.2: Class diagram showing airplane instantiation using a factory design pattern.

- **Flyweight Structural Pattern**. To reduce memory usage [19], the flyweight pattern has been employed when displaying the airplane icons for airplanes in the airport simulator. This has been done by loading and storing the image file once (using a static constant) (**Figure 4.3**), which is then available for all instantiated airplane objects to display, rather than re-loading and storing multiple copies.

```
// load once and store statically for all planes.
private static final String airplaneSpritePath = "../assets/rsz_1heavyspritet.png";
private static final Image airplaneSprite = imageLoad(airplaneSpritePath);
```

Figure 4.3: Code snippet for loading and storing the airplane icon (in ImageLoader.java).

4.4 Application Design

For the design of the application from a more 'frontend' perspective, it was decided that a main menu which allows the user to select whether they wish load an airspace or airport would suffice. After selecting between the two options, they are able to choose which airport, or which airspace to load with more buttons along with a 'Back' button if they change their mind (**Figure 4.4**).

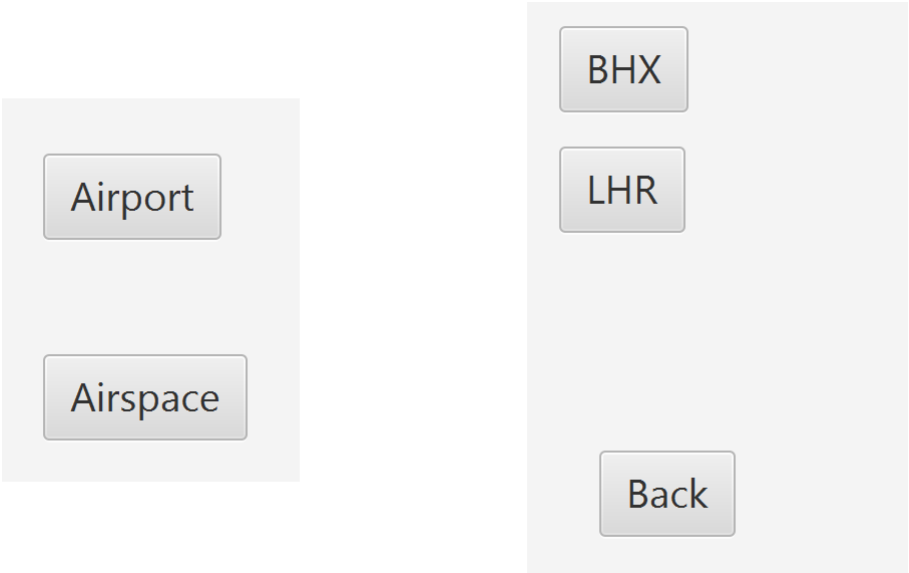


Figure 4.4: Design of main menu showing choice of environment (left), and choice of airports (right).

From the perspective of the user, the software comes in two main parts – the airport simulator and the airspace simulator. However, the system is just one simulator which can simulate both traffic at an airport, and within an airspace. This is inherently why the two simulators are extremely similar in appearance and behavior. The appearance of the simulators is relatively simple (**Figure 4.5** and **Figure 4.6**), with most of the screen taken up by the simulation view, and on the left is where the mini map is (airport simulator only) and the controls. A good design aspect of the simulator is that the user can return to the menu (button in top-left) without the need to close and restart the application.

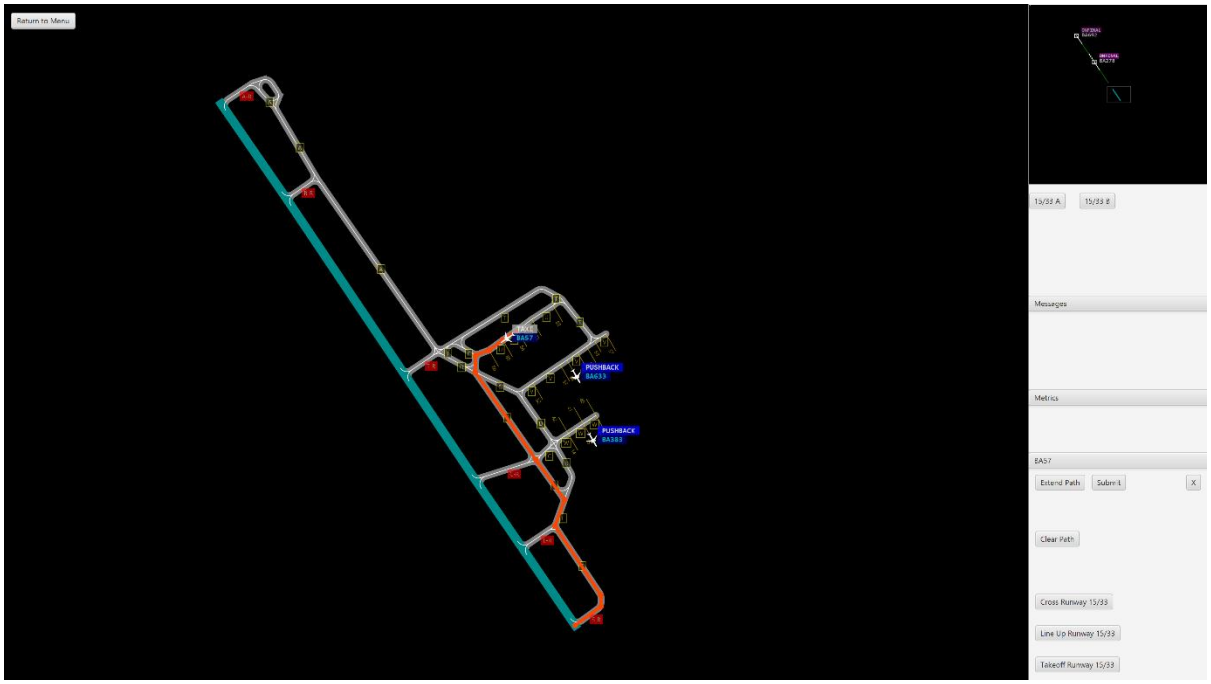


Figure 4.5: Screenshot of airport simulator.

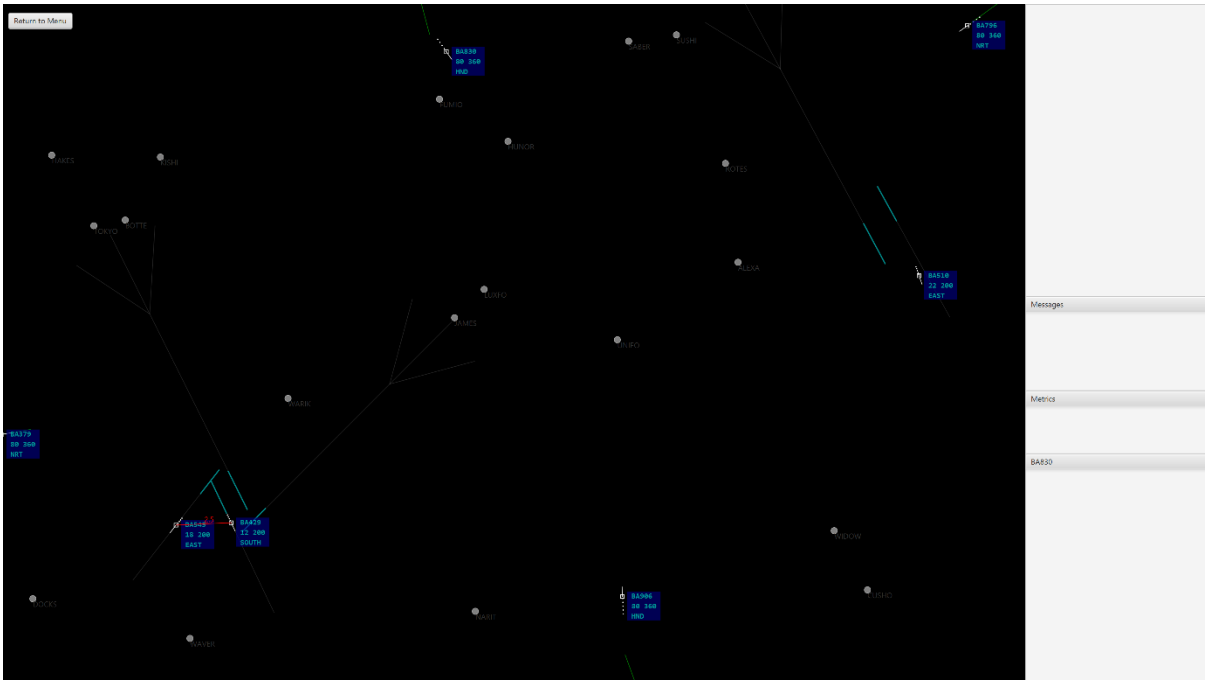


Figure 4.6: Screenshot of airspace simulator, with Tokyo Haneda (HND) in the left, and Tokyo Narita (NRT) on the right.

The design of the control buttons can be seen with a closer look at this area. The design of this part of the application is very simple – buttons are placed within the space in a sensible layout. The accompanying text on the button for the airspace simulator shows the aircraft's current altitude, heading and speed for quick reference.

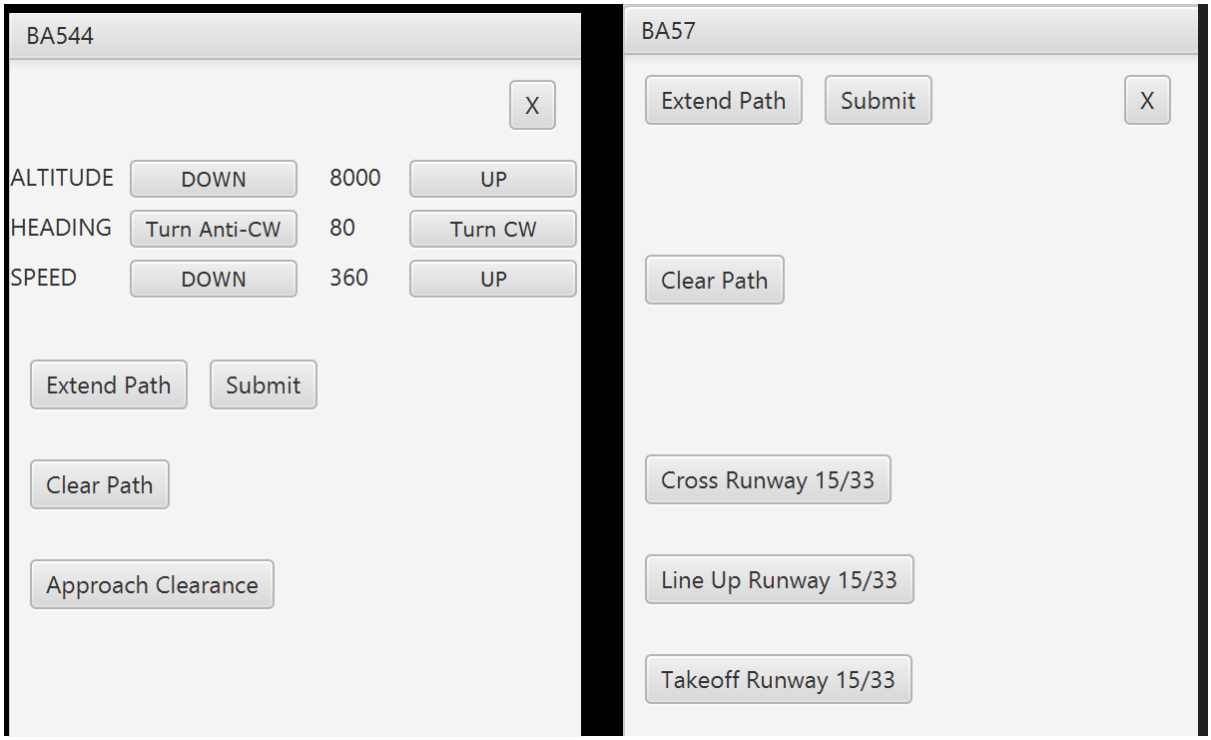


Figure 4.7: Screenshot of airplane controls in airspace simulator (left), and airport simulator (right).

The more niche aspects of the simulator design are discussed in **5 Technical Implementation**, and particularly in **5.1.9 Usability (NFR-2)**.

5 Technical Implementation

This chapter will focus on describing the main implemented features, with a particular focus on notable technical achievements and features offering an above minimum quality experience to the user. Where appropriate, a diagram, a screenshot from the system, or a code snippet may accompany the discussion. In the interest of an appropriately sized final report, this chapter will not be an exhaustive list of every functionality of the project's code; please inspect the code and the accompanying comments for a detailed understanding of specific classes or functions.

5.1 Implementation of Requirements

Since the original requirements underwent heavy modification, this sub-section will focus on the final requirements (**2.3**) to discuss how the requirements were met with the implementation.

5.1.1 Simulating the 3D space – (Requirement 1A)

The 3D space is simulated and represented to the user in a 2D manner with ease. This is achieved using a JavaFX scene (which shows as a window), with a region consisting of 1650 units high and 1080 units wide being the space for aircraft to be displayed on-screen. Each plane which needs to be displayed then has a pair of real numbers (shown in **Figure 5.1**) as a coordinate to represent its on-screen position. Planes in the airspace simulator also have an altitude number to represent their position in the vertical dimension. Sadly, the altitude of aircraft cannot be visually represented on a 2D screen, but in the next requirement's implementation (**5.1.2**) the altitude is displayed to the user.

```
protected abstract class Airplane {

    Point2D position;

    AircraftValue.Value heading;
    AircraftValue.Value speed;
```

Figure 5.1: Code snippet of the Airplane class having a Point2D JavaFX object for its position, and a heading value (in AircraftController.java).

The advantage of using a Point2D object instead of two integers to represent the aircraft's position, is that the Point2D object has methods which make it extremely useful to do some simple geometry manipulation when dealing with other Point2D objects [20].

5.1.2 Correctly representing planes on-screen – (Requirement 1B)

Aircraft are represented on-screen using JavaFX shape elements. For example, in **Figure 5.2** the representations of both a ground aircraft and in-flight aircraft which are in the airport and airspace

simulators respectively are shown. The ground aircraft representation consists of a JavaFX image, and two JavaFX labels to show a plane icon, a status tag, and the flight number. The in-flight aircraft representation consists of a JavaFX line and JavaFX square for the plane and its heading, five JavaFX circles acting as dots to show its recent path as a 'trail', and finally a JavaFX label to display the information of the aircraft such as altitude, speed, flight number and destination.

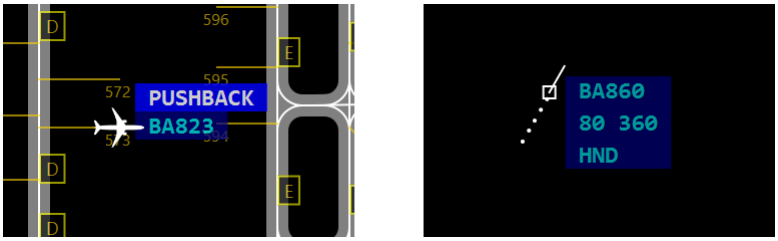


Figure 5.2: Aircraft representation, at the airport (left), in-flight within an airspace (right).

Specifically in the example shown in **Figure 5.2**, the in-flight aircraft is flying at Flight Level 80, and with 360 knots of velocity. The heading (direction) of the plane is visibly clear and its destination is HND (Tokyo Haneda International Airport) [21].

As stipulated in the final requirements 1b-2 through to 1b-4, the values associated with the aircraft must also be simulated as well as displayed. This is achieved through some basic mathematical functions. For example, the change in live altitude, live heading and live speed simply is performed for each plane by increasing or decreasing these numbers until they reach a 'target' value which may be specified by a user input. This way, a plane flying at a live heading of 060 may turn by a few degrees each second until it gets to a 'target' heading of 180. Finally, the position of each aircraft at the end of each 'time tick' is updated using the live values in the following calculations (**Figure 5.3**).

$$t - time\ elapsed\ (seconds)$$
$$h - live\ heading\ (degrees)$$
$$s - live\ speed\ (knots)$$
$$C - knots\ to\ \frac{pixel}{second}\ conversion\ constant\ [specific\ to\ each\ map]$$
$$\Delta x - change\ in\ x\ coordinate\ (pixel)$$
$$\Delta y - change\ in\ y\ coordinate\ (pixel)$$

$$\Delta x = C \cdot \sin\left(h \cdot \frac{\pi}{180}\right) \cdot s \cdot t$$
$$\Delta y = C \cdot \cos\left(h \cdot \frac{\pi}{180}\right) \cdot s \cdot t$$

Figure 5.3: Calculations to find the change in coordinates of a plane for each 'time tick'.

We can see how the calculations in **Figure 5.3** have been used to implement this into the solution (**Figure 5.4**). The solution looks slightly different to the raw calculations as some of the calculations have been extracted out into their own functions where they may be used in many other parts of the system - reducing repeated code.

```
// calculate and update the new position of the aircraft, dep. on how many seconds have elapsed since last 'tick'
private void movePosition(double secondsToCommit) {
    double xPixels = MyMath.knotsToPixelsPerSecond(MyMath.resolveHorizontal(heading.live, secondsToCommit * speed.live));
    double yPixels = MyMath.knotsToPixelsPerSecond(MyMath.resolveVertical(heading.live, secondsToCommit * speed.live));
    position = position.add(xPixels, yPixels);
}
```

Figure 5.4: Code snippet of movePosition in the Airplane class (in AircraftController.java).

After the position has been updated for the current 'time tick', the method updateVisualElements() in the class AircraftVisualElementsAirborne (in AircraftVisualElementsAirborne.java) or AircraftVisualElementsGround (in AircraftVisualElementsGround.java) is then used to correctly move all the visual JavaFX components to the aircraft's new position on-screen.

Finally, to satisfy requirement 1b-6, a mini-map was implemented on the airport simulator for aircraft that had departed or were arriving at the airport. This uses the coordinate system to store the position of each aircraft implemented for requirements 1A (**5.1.1**), but uses a few additional coordinate transformations to display the aircraft on a much smaller scaled map. The visual result of this implementation can be seen in **Figure 5.5**.

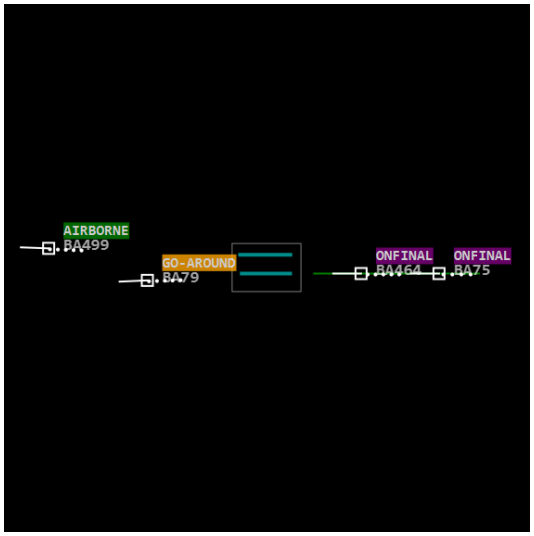


Figure 5.5: Mini map showing airborne aircraft departing or arriving at airport (airport simulator).

The mini map in **Figure 5.5** shows four aircraft with their flight number and their status (e.g., GO-AROUND). It also shows the aircraft pointer with trail, alike to the representation of aircraft in the airspace simulator (right of **Figure 5.2**). This mini map serves as a useful tool to see which aircraft are leaving and arriving at the airport and does not show unnecessary details such as all the airplanes on

the ground which are visible on the main screen. A miniaturized replication of the runways and main screen view window is also displayed in the center of the mini-map for orientation.

5.1.3 Detecting whether a violation of ATC rules have occurred (Requirement 1C)

Minimum Separation

One ATC rule the simulator detects to see if it has been broken is the rule of minimum separation as in requirement 1c-1. Mathematically, this is a very simple implementation. The coordinate each plane is used to check the <u>lateral</u> distance between each pair of planes using the Pythagorean equation.

$$(x_1, y_1) - position\ of\ plane1$$
$$(x_2, y_2) - position\ of\ plane2$$
$$d - lateral\ distance\ (pixels)$$
$$d = \sqrt[2]{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Figure 5.6: Equation to find lateral distance between two planes at two points.

For each pair of planes, this lateral distance is converted to the nautical miles equivalent distance to see if it is within 3nm. If the pair of planes are also strictly less than 1,000ft apart vertically then the planes are deemed to be *too close* – a violation of the ATC rule. When planes are (nearly) *too close*, a JavaFX line will appear on-screen with a JavaFX label to state the exact lateral distance (**Figure 5.7**).

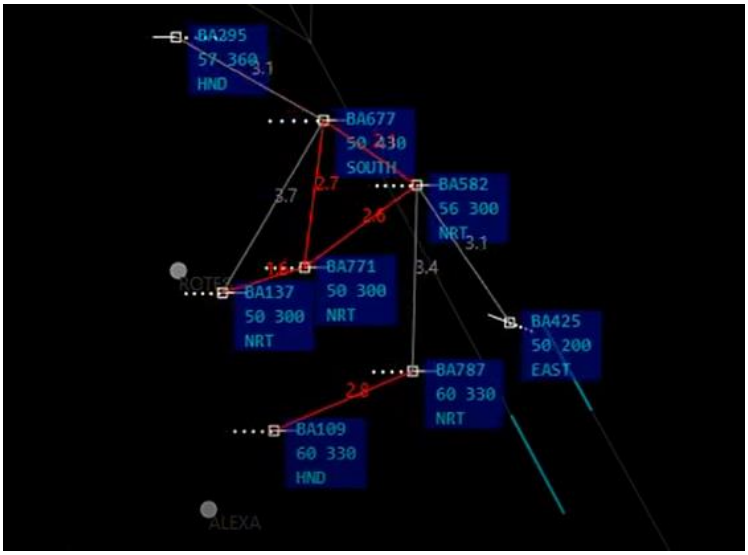


Figure 5.7: A screenshot of the airspace simulator showing a dangerous minimum separation situation arranged deliberately.

A particular strength of the visual separation system is the cleanliness and simplicity of the visual design. As in **Figure 5.7** a line and label are present for when two planes are within minimum separation in a red color. Not only does red visibly stand out and attract the controller's attention, but

it is also an appropriate color as it can convey the meaning of danger [22] – the imminence of airplanes colliding in the sky. There is also a grey color for when two planes are almost within minimum separation, acting as a visual caution before any violation occurs. One should note that where planes appear laterally close but have no line between them, they are vertically separated sufficiently (e.g., BA425 and BA787).

Within the system, there is an attempt to reduce the overhead of this distance checking process. Given the set of aircraft $A = \{a_1, a_2, \dots, a_n\}$, all the distinct pairs of planes are in the set $P = \{(a, a') \in A^2 : a < a'\}$, for which $|P| \in O(n^2)$. Therefore, a naïve approach would be to check the distance for all $O(n^2)$ pairs of planes on every 'time tick' of the simulator. However, a method to reduce this overhead is to calculate the distance between all $O(n^2)$ pairs of planes (a full check) much less regularly (e.g., once every 15 seconds). Using this information from each full check, the system would partition the set of pairs of planes F into two sets:

$$X = \{p \in P : the\ pair\ p\ is\ within\ 3\ times\ the\ minimum\ separation\}$$
$$\bar{X} = F \backslash X$$

This way, the set X contains all pairs of planes which are at risk of being *too close* before the next full check, and the set $\bar{X}$ contains all pairs of planes which are deemed to not be at risk of becoming *too close* before the next full check. The system can then check the exact distances of all pairs of planes in X on each 'time tick' of the simulator to provide real-time data on their separation. This is more efficient as 3 times the minimum separation is a small part of the screen and therefore there only remains a fraction of all pairs of planes; effectively we have $|X| \ll |P|$. The advantage here is that the time overhead on each 'time tick' of the simulator is greatly reduced by not wasting processing time on pairs of planes which have no risk, while no pairs of planes at risk are missed out since a full check if conducted every so often. In summary, although asymptotically the running time is still $O(n^2)$, the running time is significantly lowered.

Runway Occupation Detector

The other ATC rule for which compliance is checked within the system is the rule of not having multiple aircraft on the runway at the same time such that a risk of collision exists. Like the minimum separation feature, this is not a very complicated implementation.

A plane is an *active occupant* of the runway if the plane is on the runway for any purpose. Each runway stores a list of its *active occupants* (**Figure 5.8**). The important detail with this is the cardinality of the list of *active occupants* for each runway; how many planes there are on the runway.

```
// Should only ever be size <= 1
LinkedList<AircraftController.Airplane> activeOccupants = new LinkedList<>();
```

Figure 5.8: Code snippet showing the LinkedList to store the *active occupant* aircraft, as a property of the Runway object (in EnvironmentController.java).

More formally, for any runway the number of *active occupants* will increase by one when a plane begins to cross, touches down (landing), or begins takeoff roll (takeoff). Conversely, the number of *active occupants* will decrease when a plane exits the runway or lifts off. The number of *active occupants* should not exceed 1 if there is a landing aircraft involved. If such a case occurs, a warning will appear showing which two planes may be in conflict, shown in **Figure 5.9**. The change in the number of *active occupants* is achieved simply by adding or removing planes from the LinkedList when necessary.

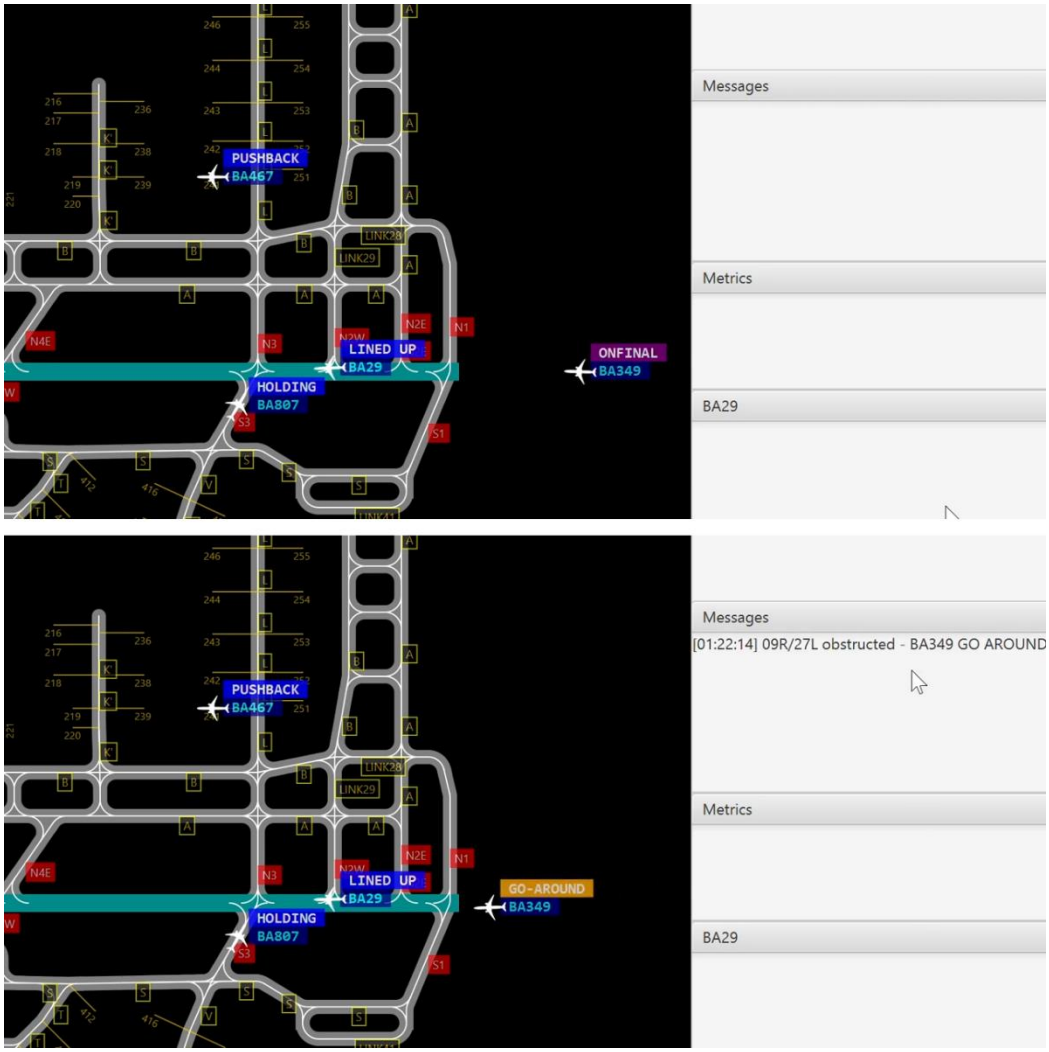


Figure 5.9: Plane (BA349) attempting to land on an occupied runway, leading to a go-around and message alert, before (top), and after (bottom).

This is a good safety feature as it alerts the controller to the close-call and represents real-world procedure since a pilot would abort a landing attempt if the runway was known to be occupied [23]. A written warning is also displayed in the messages panel with a timestamp, so the user knows when it occurred.

5.1.4 Representation of non-aircraft entities (Requirement 1D)

It is necessary to represent and implement the non-aircraft entities to have a proper simulator. This included taxiways, runways, and parking gates for the airport simulator, and included runways, waypoints, and approach paths for the airspace simulator.

For the airport simulator, the sensible approach to implement the airport entities was using graph theory. The notable advantages of this graph theoretic implementation are described in **5.2.1**, however this section will focus on the implementation details.

In graph theory, the simplest and most common way to define a graph is to have a set of points and a set of lines connecting pairs of points. This can be formalized to a set of vertices and a set of edges respectively [24]. The implementation of an airport network will adapt this traditional definition of a graph into a less restrictive form. By adapting each edge which connects a pair of vertices to become a 'multi-edge' which goes through multiple vertices in series, it offers an advantage where it becomes easier to design a taxiway which naturally goes through many points from one side of the airport to another; just like how a road for cars does not just connect two points. An illustration of this concept and how it is encoded in the text file which stores an airport is shown in **Figure 5.10**.

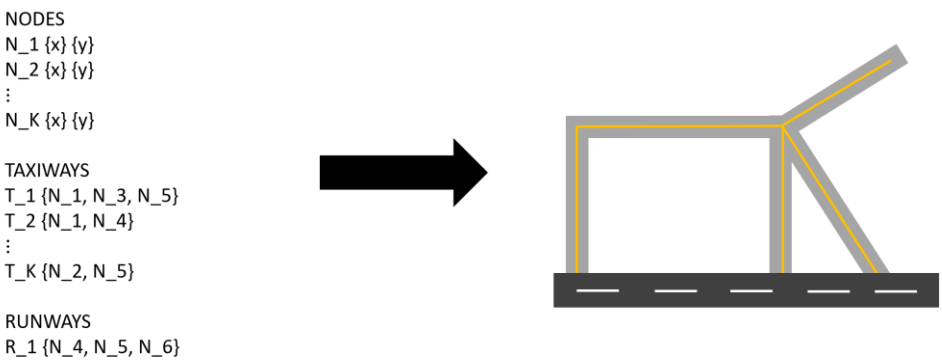


Figure 5.10: Illustrated concept of the airport encoding in the text file, and a visual graph of the airport.

To take this implementation one step further, we can define the set of runways distinctly from the set of taxiways since they have different functionalities within the system. A set of ramps are also implemented, which are ordinary taxiways but just are incident to a runway. Finally, up to two

parking gates are defined with each node to come off the taxiways. This lends us to a defined format to fully encode an airport shown in **Table 5.1**.

Airport Encoding
{real: nauticalMilesPerPixelScale}
NODES
{String: nodeName1} {int: xPosition} {int: yPosition}
{String: nodeName2} {int: xPosition} {int: yPosition} {int: parkingGateNumber},{int: direction},{int: length}
⋮
{String: nodeNameN} {int: xPosition} {int: yPosition}
TAXIWAYS
{String: taxiwayName1} {node_name},{node_name},…,{node_name}
{String: taxiwayName2} {node_name},{node_name},…,{node_name}
⋮
{String: taxiwayNameT} {node_name},{node_name},…,{node_name}
RAMPS
{String: rampName1} {node_name},{node_name},…,{node_name}
{String: rampName2} {node_name},{node_name},…,{node_name}
⋮
{String: rampNameR} {node_name},{node_name},…,{node_name}
RUNWAYS
{String: runwayName1} {node_name},{node_name},…,{node_name}
{String: runwayName2} {node_name},{node_name},…,{node_name}
⋮
{String: runwayNameW} {node_name},{node_name},…,{node_name}
Parking gate encoding in blue – up to two allowed per node.

Table 5.1: Format for airport encoding.

Next, the system must be able to parse this format and construct the airport each time the simulator is started. This is because this airport encoding is permanently stored in text files which makes them easy to access without needing to navigate the code. The system will line-by-line read the file and interpret the data in the airport encoding, with the use of labels such as “NODES” or “TAXIWAYS” etc., to allow the system to know what entity encoding is on the next line. This process is illustrated in **Figure 5.11**.

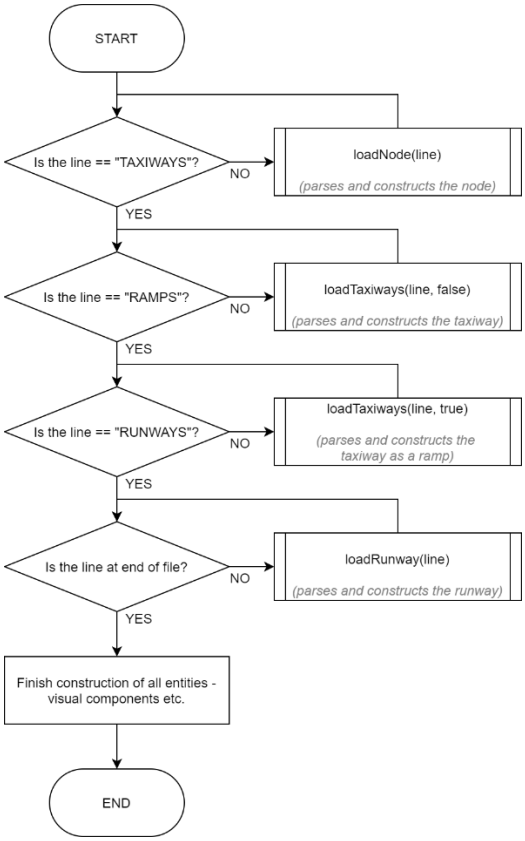


Figure 5.11: Flowchart to show the line-by-line airport construction logic for the method loadAirportFile() (in EnvironmentContoller.java).

Similarly, airspaces are encoded and constructed in a similar way with different entities. Airspace entities include runways (belonging to an airport), waypoints, and entry points; directions from which an arriving plane can fly onto the screen. We will omit the flowchart which shows the logic for this as it is almost identical to that of the flowchart for airports (**Figure 5.11**), although the encoding format is non-trivially distinct and is therefore shown in **Table 5.2**.

Airspace Encoding
{real: nauticalMilesPerPixelScale}
RUNWAYS
{String: airportName1} {int: runwayStartX},{int: runwayStartY} {int: runwayEndX},{int: runwayEndY} {arr.} {dep.}
{String: airportName2} {int: runwayStartX},{int: runwayStartY} {int: runwayEndX},{int: runwayEndY} {arr.} {dep.}
{String: airportName2} {int: runwayStartX},{int: runwayStartY} {int: runwayEndX},{int: runwayEndY} {arr.} {dep.}
⋮
{String: airportNameA} {int: runwayStartX},{int: runwayStartY} {int: runwayEndX},{int: runwayEndY} {arr.} {dep.}
WAYPOINTS
{String: waypointName1} {int: xPosition} {int: yPosition}
{String: waypointName2} {int: xPosition} {int: yPosition}
⋮
{String: waypointNameW} {int: xPosition} {int: yPosition}
ENTRIES

{int: heading} {int: heading} … {int: heading}
Airport names in blue are allowed to be non-distinct; there can be multiple runways to an airport. The tokens {arr.} and {dep.} are 0 or 1 for whether a runway should be used for arrivals and for departures (1 = true).

Table 5.2: Format for airspace encoding.

Unfortunately, the construction of these encoding files is relatively cumbersome; one must manually type in all the coordinates of each entity of an airport or airspace, which can take a few hours to complete. This is especially the case when one wishes to make an airport or airspace encoding to scale by using real-life charts or maps to generate the positions of the entities. However, these text files offer the advantage of being easy to edit and portable; one may send a text file to another person to give them a new airport.

So far this provides a good implementation of how to construct the encoding for the airport or airspace and how the system reads it, but we must finally discuss how these are stored in the system and particularly the object-oriented nature of this part. The relevant objects to represent (either part of or the entirety of) the airport entities are: NavigationNode, TaxiwaySegment, Taxiway, RunwaySegment, Runway, and Gate. For the airspace entities these are: AirspaceRunway, Waypoint, and Entrance. All these objects' classes are stored in EnvironmentController.java. Currently the list of classes does not provide a full explanation of how the entities are represented, and therefore this is detailed further in **Table 5.3** and **Table 5.4**.

Class	Description	Inherits…	Contains…
NaivgationNode	A point in 2D space.	-	A Point2D JavaFX object. A list of TaxiwaySegments which are incident to this node. A list of RunwaySegments which are incident to this node.
TaxiwaySegment	An edge connecting two nodes for a taxiway.	-	Two NavigationNode objects for start and end points.
Taxiway	A full taxiway through all its edges.	-	A list of TaxiwaySegments which are part of the taxiway.
RunwaySegment	An edge connecting two nodes for a runway.	TaxiwaySegment	(Items contained by TaxiwaySegment through inheritance)
Runway	A full runway through all its edges.	Taxiway	(Items contained by Taxiway through inheritance) Two NavigationNode objects for the start and end points of the entire runway.
Gate	An edge to represent a parking gate.	TaxiwaySegment	(Items contained by TaxiwaySegment through inheritance)

Table 5.3: Description of classes used to represent airport entities.

Class	Description	Contains…
AirspaceRunway	An edge representing a runway of an airport.	Two Point2D JavaFX objects for the start and end points of the runway.
Waypoint	A point in 2D space.	A Point2D JavaFX object.
Entrace	A point for planes to arrive in the airspace, defined by a heading from the center of the screen.	A Point2D JavaFX object for the point on the screen's edge. A Point2D JavaFX object between the first point and the center of the screen (used to draw a line through these two points).

Table 5.4: Description of classes used to represent airspace entities.

With this construction of java classes to represent the airport and airspace entities, it makes it possible to give these objects many other properties and methods (omitted from **Table 5.3** and **Table 5.4** as they are not relevant to how the classes relate to one another). These other properties and methods provide good functionality to the system, as for example, most objects have their own function to construct and display the visual elements on the screen. Displaying the visual elements on the screen is mostly straightforward and limited to specifying basic parameters of shapes (such as position, radius, or color etc.), but can sometimes be more complicated and require heavy geometric calculations. For example, the display of the approach guidelines, taxiway curves, and parking gate labels required geometric manipulation such as interpolation or extrapolation of points and trigonometry to correctly display them (**Figure 5.12**).

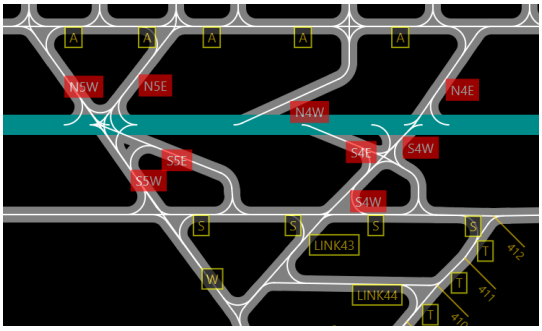


Figure 5.12: Screenshot of airport simulator showing taxiways intersecting at a variety of angles.

In the case of the taxiway curves in **Figure 5.12**, the only pre-defined positions are the nodes of the airport graph. All taxiway and centerline arcs drawn are geometrically calculated on these stored positions and have a constant radius as in real-life, as shown in **Figure 5.13**.

Figure 5.13: Taxiway centerlines with constant radius at London Heathrow airport [25].

5.1.5 Issuing and executing control commands for the aircraft from the user (Requirement 1E)

Before discussing the implementation of the ATC commands for the aircraft, there will firstly be some remarks on the structure of the implementation for this requirement. Considering that a basic GUI must be accompanied with the more 'backend' functionality, the decision was taken not to implement this in the AircraftController class (in AircraftController.java), but to make two new classes AirborneAircraftControls (in AirborneAircraftControls.java), and GroundAircraftControls (in GroundAircraftControls.java) for the airport and airspace simulator respectively, which both inherit the class Controls, and both implement the interface Controllable. This enabled the code to be segmented better into related areas of responsibility and thus aids the maintainability of the software.

Additionally, only relevant buttons are displayed for each aircraft to make the GUI simpler and more interpretable (e.g., there is no button for take-off for a plane coming in to land). Some of these buttons are shown in **Figure 4.7** and **Figure 5.14**.

Changing altitude, speed and heading for airborne aircraft.

Aircraft in the airspace simulator must be able to be controlled using altitude, heading and speed (AHS). To implement this in the system there are three sets of JavaFX buttons for each of these values which show up when a user clicks on a plane. The buttons are labelled with "DOWN" or "UP", except for the heading controls where this labelling would not be very intuitive and therefore are labelled with "Turn Anti-CW" and "Turn CW" (**Figure 5.14**).

Figure 5.14: Screenshot of AHS controls for aircraft in the airspace simulator.

Since the functionality of all three sets of controls (altitude, heading, and speed) are very similar, they have been implemented using a superclass AirplaneVariableControl, and subclasses AltitudeControl, HeadingControl, and SpeedControl – all in AirborneAircraftControls.java. This helps improve modularity and reusability of the code. Additionally, it enhances flexibility as it is possible to customize each subclass – the HeadingControl class facilitates the cyclic nature of headings (i.e., 359 and 000 being 1 degree apart).

Each of these controls operates under different parameters as the numerical values have very different ranges. For example, it would make no sense to increase both the altitude and speed by 10 at a time; 10ft is miniscule. Therefore, the class associated with each value (e.g., AltitudeControl for altitude) specifies parameters for the controls to operate under. In the case of altitude, the value must stay

between 3,000ft and 15,000ft, and only change by 1,000ft as common practice in real-life [26] (**Figure 5.15**).

```
public class AltitudeControl extends AirplaneVariableControl {

    // parameters for altitude control
    private int upperBound = 15000;
    private int lowerBound = 3000;
    private int increment = 1000;
    private int decrement = -1000;
```

Figure 5.15: Code snippet for the altitude control parameters (in AirborneAircraftControls.java).

Finally, when the user submits these changes in AHS to the aircraft, the target AHS values of the plane will update and the aircraft will change its flight (e.g., by slowing down). The change in flight is achieved by additional classes which manage the AHS values: a superclass Value, inherited by subclasses Altitude, Heading, and Speed in AircraftValue.java (**Figure 5.16**).

```
public static class Altitude extends Value {
    private static final int PERIOD = 1;
    protected static Double perSecondRate = 50.0;   // default rate

    // constructors
    public Altitude(double live, int target) { super(live, target); }
    public Altitude(double live) { this(live, target:0); }
    public Altitude() { this(live:0, target:0); }

    @Override
    public void change(double secondsToCommit) {
        // change at default rate
        change(secondsToCommit, perSecondRate);
    }

    public void change(double secondsToCommit, double changeQuantity) {
        // change at parameter specified rate
        super.change(secondsToCommit, changeQuantity, PERIOD);
    }
}
```

Figure 5.16: Code snippet of Altitude class to handle the altitude of a plane (in AircraftValue.java).

These subclasses Altitude, Heading and Speed also operate under specific parameters. The natural rate of change in the AHS values are all different, and as shown in **Figure 5.16** is 50ft per second for altitude, which is close to the typical ascent or descent rate of a Boeing 787-8 [27]. These subclass parameters dictate the rate at which each of the AHS values will change for an aircraft in the live simulation.

Clearing planes for final approach.

In the airspace simulator, planes can approach a runway and finally land at an airport. The user can give clearance for a plane to approach if the plane is:

- within the approach guideline markers (faint grey – left-bottom of **Figure 5.18**) for a runway of the destination airport,

- at altitude 5,000ft or less,
- at speed 240 knots or less, and
- is flying towards the runway.

```
// check all runways
for (EnvironmentController.AirspaceRunway runway : runwaySet) {

    // skip if the runway is not part of the destination.
    if (!runway.airport.equals(airborneAirplane.destination)) { continue; }

    // find distance and heading difference to the point that is 8 naut. miles from runway
    Point2D NM8_ILS = runway.ILS_8NM;
    double distanceNM = MyMath.distance(airplanePosition, NM8_ILS) * EnvironmentController.nmPerPixel;
    double headingDifference = MyMath.headingDifference(airborneAirplane.heading.live,
                                                        MyMath.findHeading(airplanePosition, NM8_ILS));

    if (distanceNM > 5) { continue; }

    if (headingDifference > 50) { continue; }

    if (airborneAirplane.altitude.live > 5000) { continue; }

    if (airborneAirplane.speed.live > 240) { continue; }

    // Approach for this runway is successful
    nominatedApproachRunway = runway;
```

Figure 5.17: Code snippet for code to check if the plane has met criteria to begin final approach and nominate an approach runway (in AirborneAircraftControls.java).

These criteria are to ensure realistic approaches, and otherwise there would be no challenge to the user. After a plane has successfully been cleared for approach, the system will change the color of the aircraft's JavaFX label (its information tag) (**Figure 5.18**) to show the user that they do not need to control the plane anymore – the plane will guide itself to the runway.

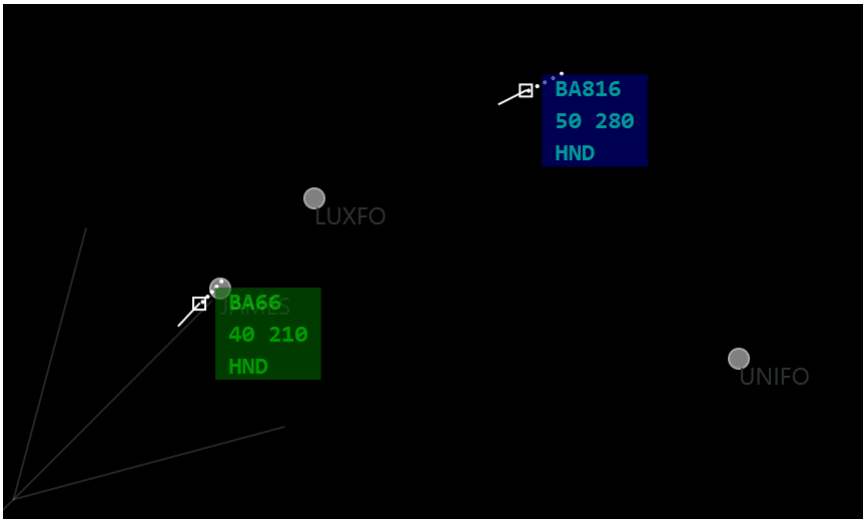


Figure 5.18: Screenshot of airspace simulator showing BA66 cleared for final approach to HND (Tokyo Haneda Airport).

Should a user's final approach clearance attempt not meet all the criteria for any runway, there will be no nominated runway in the code for **Figure 5.17**, and the code in **Figure 5.19** will display a message to the user.

```
// Approach clearance attempt failed if no nominated runway.
if (nominatedApproachRunway == null) {
    AircraftController.messagesManager.pushMessage("INVALID APPROACH for " + airplane.flightID);
    return;
}
```

Figure 5.19: Code snippet to check whether a plane has failed the approach attempt (in AirborneAircraftControls.java).

Taxiing aircraft at the airport.

Enabling planes to follow a precise path requires some careful implementation to ensure it works properly. The implementation gives all planes a queue of nodes to visit along their path, and planes always travel toward the next node along the path. Once a plane is close enough to the current target node, it will switch its direction of travel to the next target node by popping it off the stack of nodes for the entire path. This logic is illustrated in **Figure 5.20** and an example is visualized in **Figure 5.21**.

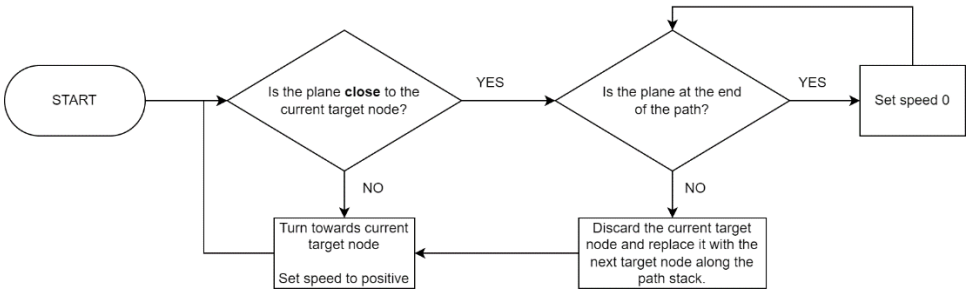


Figure 5.20: Flowchart to illustrate logic for planes following a path (for the airport simulator).

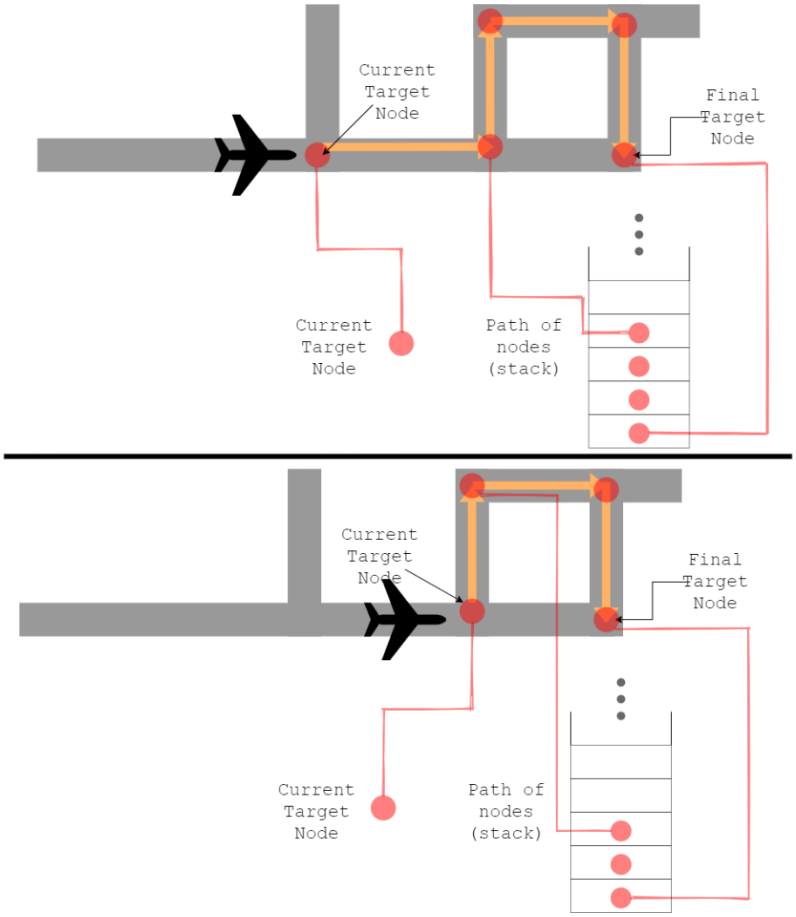


Figure 5.21: Visualized example of how the current target pointer and path stack works between passing a target node, before (top) and after (bottom).

Since this path following algorithm does not terminate, whenever a plane's path needs to be extended, the ordered set of nodes are added to the bottom of the stack (traditionally one cannot add to the bottom of a stack, but Java data structure implementations make this more flexible; the list can be treated both as a stack and a queue). If a plane's path needs to change then it is possible to empty this stack, meaning the plane will not go beyond the current target and can wait for its stack to be filled up with nodes again. This ability to clear then change the path is helpful as the user can recover from a mistake or if they change their mind for where the plane should go.

So far this provides a good framework for letting planes navigate around the network of nodes. However, it is still necessary to implement user-controls to dictate where the planes should go. The routing system implemented allows the controller (the user) to select a plane, and either clear the path, or to extend the current path to route it to a new final target. When extending an aircraft's current path ending at vertex v_1 to a new vertex v_2, the A* Search algorithm is used to determine the shortest path from v_1 to v_2. This method of allowing the controller to extend the path of an aircraft is helpful because:

- it is not too cumbersome; the controller does not need to select every node the plane will travel through between v_1 to v_2 thanks to the A* Search algorithm, and
- it allows precise control; the controller can select as many nodes as necessary to extend the path multiple times if they need the aircraft to follow a particular route (e.g., to avoid other traffic).

One crucial consideration to ensuring this implementation is sensible is the choice of the search algorithm. The A* Search algorithm was chosen as it greedily chooses the vertex that is more likely to lead towards the target fastest since it uses an admissible heuristic (our heuristic is the remaining distance as the crow flies), leading to the first path found by the A* Search algorithm to be the optimal path [28]. This is implemented in Explore.java. This is much better than alternative search algorithms such as Dijkstra's Search algorithm, which although works similarly to A* Search through the greedy property of expanding the best node in the frontier, does not utilize an admissible heuristic to speed up this process to expand nodes which are estimated to be closer to the target [29].

Normally, the simulator will handle planes taxiing around parts of the airport in an uneventful manner. However, as a special type of taxiing operation, a plane can be made to *line up* on a runway – this is the immediately prior step before a plane takes off, and sometimes may be skipped when traffic is moving fast. In the airport simulator, when a departing plane's taxiing path intersects with a runway, the option is available to have the plane *line up* at the first point it contacts a runway. For this reason, the controller must not click the LINE UP button too early if the plane needs to cross another runway to get to the departure runway, otherwise the plane will *line up* on the other runway first. In the airport simulator, the ability to *line up* planes is unavailable on arriving aircraft (planes which have landed or are landing) for obvious reasons.

The other type of special taxiing operation is for planes to *cross* a runway. When a plane *crosses* a runway, it is not supposed to use the runway for take-off at the point of crossing and should pass over it gracefully as if it were a normal taxiway. Since it could be unsafe for airplanes to make an uncommanded *cross* over a runway that may be in use by another landing aircraft, a feature is implemented in the airport simulator to have planes wait before the runway, and cross only when authorized by the user. This feature is functional for both when a plane needs to *cross* at a single point, or when the plane must *cross* by travelling down part of the runway to get to the other side. Since the movement permissions are set at a node-level, this is achieved by setting the permission of the plane's path edges to CROSS, not just for the first edge which intersects the runway, but for all following consecutive edges which intersect the <u>same</u> runway as illustrated in **Figure 5.22**.

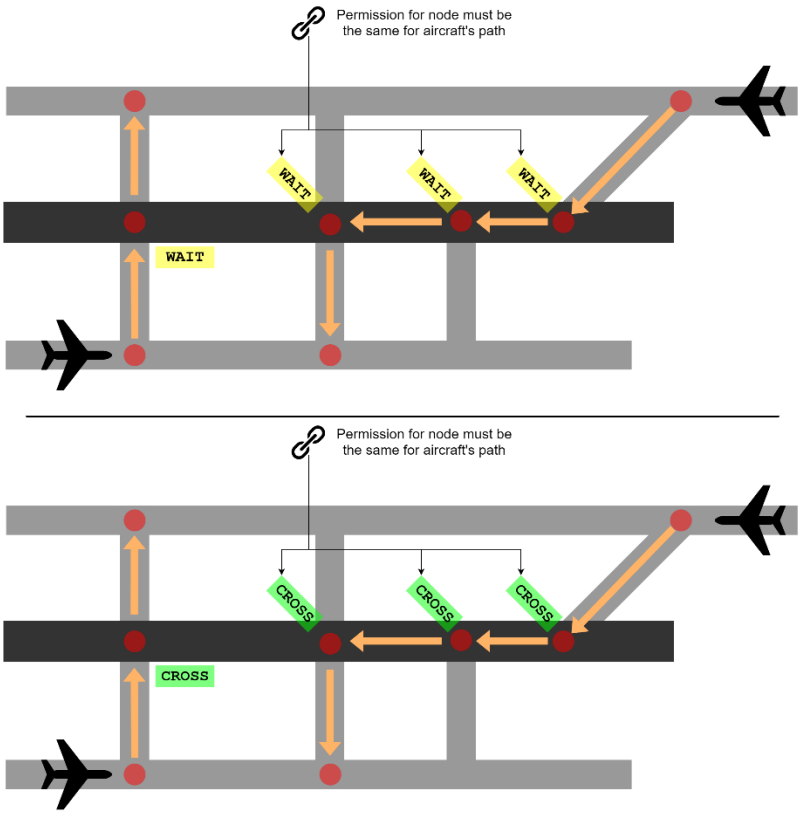


Figure 5.22: Permission set for segments of two plane's paths crossing a runway at a single point and over multiple points before authorization (top), and after (bottom).

Furthermore, if a plane is to cross multiple distinct runways over its taxiing path at various points, an authorization to cross will need to be provided for each of these runways.

Routing aircraft via waypoints.

Routing aircraft in the airspace simulator through series of waypoints is similar and a more general implementation of the airport taxiing system through a path of nodes. Instead of the aircraft being confined to taxiways acting as edges, an airborne aircraft can fly to any waypoint in the airspace. Therefore, there is no need for graph traversal, and the implementation described in **Figure 5.20** and **Figure 5.21** is used in the same way for this feature. The only difference is that the start of the path is no longer the previous node travelled but is instead the current aircraft position (**Figure 5.23**) since airborne aircraft could be in an arbitrary position.

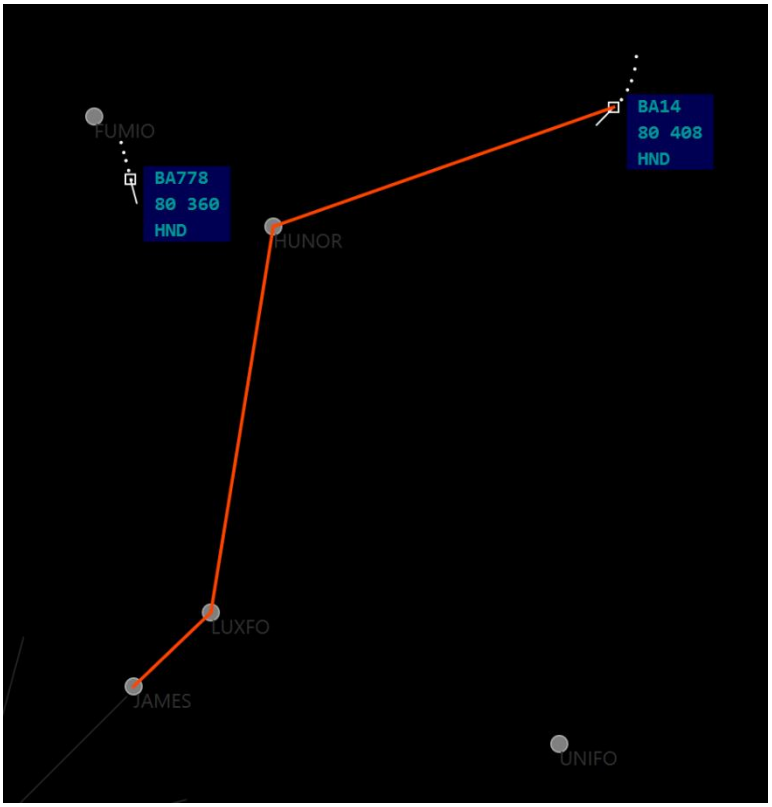


Figure 5.23: Screenshot of BA14 turning to the next waypoint on its path.

Takeoff, landing, and go-arounds.

Finally, aircraft must have the ability to arrive at and depart from an airport; to take-off and land, and in exceptional situations to abort the landing attempt; a go-around. These features were more challenging to implement, particularly because planes and their movement system within the airport simulator are normally confined to the graph theoretic network of taxiways (**5.1.4**) and therefore many adaptions were put in place to avoid issues.

Aircraft taking off was the simplest of the three. Upon providing the command to take off a plane will extend its path to the other end of the runway. However, instead of this plane taxiing, it will travel along this path at a controlled accelerating speed. The acceleration is constant, which is unfortunately not realistic, but this is an acceptable abstraction to make in the simulator since it is not worth the effort to model the precise acceleration profile of a typical aircraft on takeoff. After the aircraft reaches a specified speed of 160 knots it will become airborne and no longer be confined to the graph theoretic network of taxiways and will fly off the screen. The aircraft will only be deleted by the simulator after it flies off the mini map, which was described in **5.1.2**.

The ability for aircraft to land and go-around was more challenging. Firstly, it was necessary to have the aircraft spawn off the main screen far from the airport to be on approach before it lands. This lent to the implementation of an additional control for the user, where they can customize which runways

are available for an approach, shown by the green line (**Figure 5.24**). The buttons correspond to changing the approaches available under two constraints: there must be at least one approach open, and no runway can have an open approach from both directions – in the interest of safety. When an airplane is close to the runway at 'decision altitude', it will check to see if the user has given landing clearance. In the affirmative case, the aircraft will proceed to land and eventually put itself on the graph theoretic network of the airport at the runway, otherwise it will automatically initiate a go-around. Go-arounds put the aircraft in a mode similar to takeoff where they will fly away from the airport. In real-life an aircraft on a go-around will fly back around for another approach, but this detail is abstracted away from the simulator. Go-arounds can happen automatically if landing clearance is not given as aforementioned, or additionally can happen if the user instructs the plane to go-around.

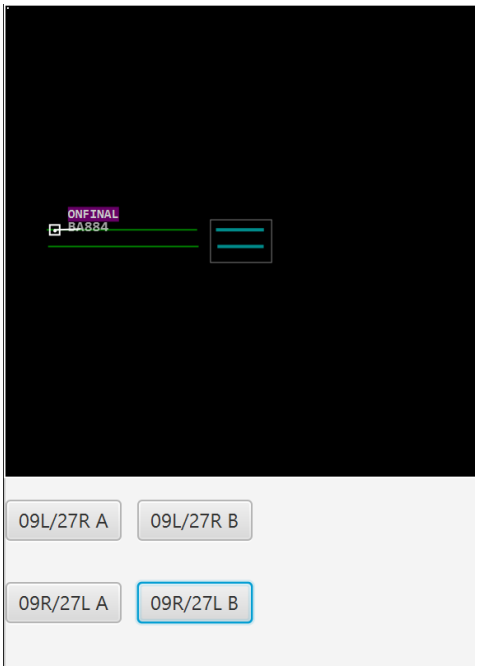


Figure 5.24: Screenshot of mini map with controls for selecting open runway approaches.

An extra feature to improve the controllability of landing planes was the ability to select the exit from a runway (**Figure 5.25**). At the point of landing, the plane will automatically add a path along the graph to some exit such that it can follow this path along the graph. This exit is randomly determined unless the user specifies an exit by clicking the green JavaFX circles <u>before</u> landing.

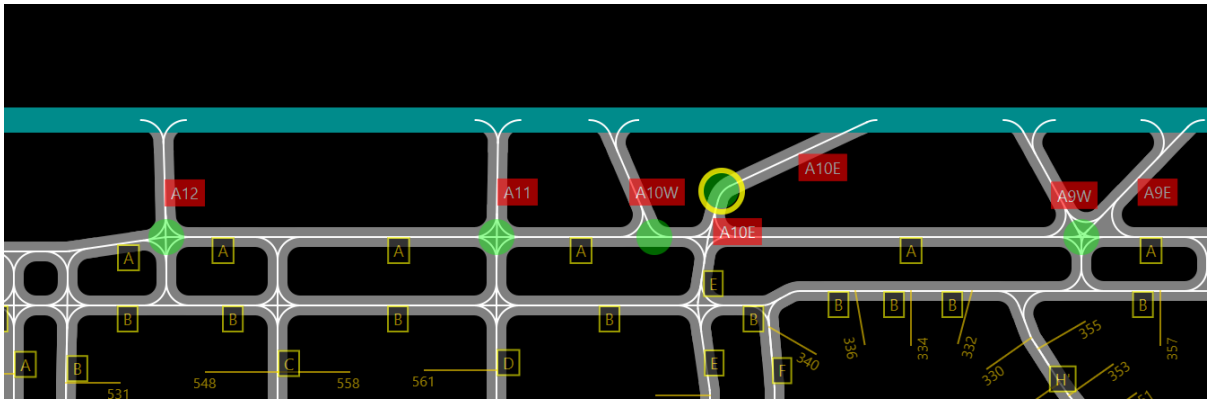


Figure 5.25: Screenshot of green runway exit selectors, with a yellow circle for the selected exit.

Finally, to implement the post-landing phase of the aircraft, a vacant parking gate is selected which is highlighted using another JavaFX circle when the arrived plane is clicked on (**Figure 5.26**). The user should direct the plane by extending its path to the taxiway adjacent to this parking gate for the plane to arrive fully at the terminal.

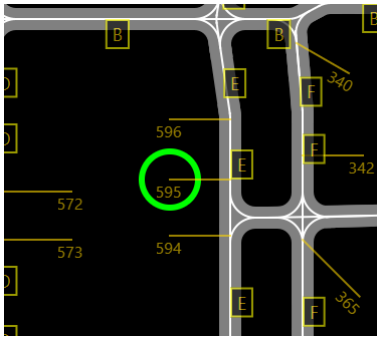


Figure 5.26: Highlighted parking gate for arrived aircraft to arrive at.

5.1.6 Tutorial Mode (Requirement 2A)

To implement the ability to start the simulator in a tutorial mode, a JavaFX CheckBox object is displayed on the main menu (**Figure 5.27**). This offers a convenient way for the user to decide whether they want to be in tutorial mode or not.

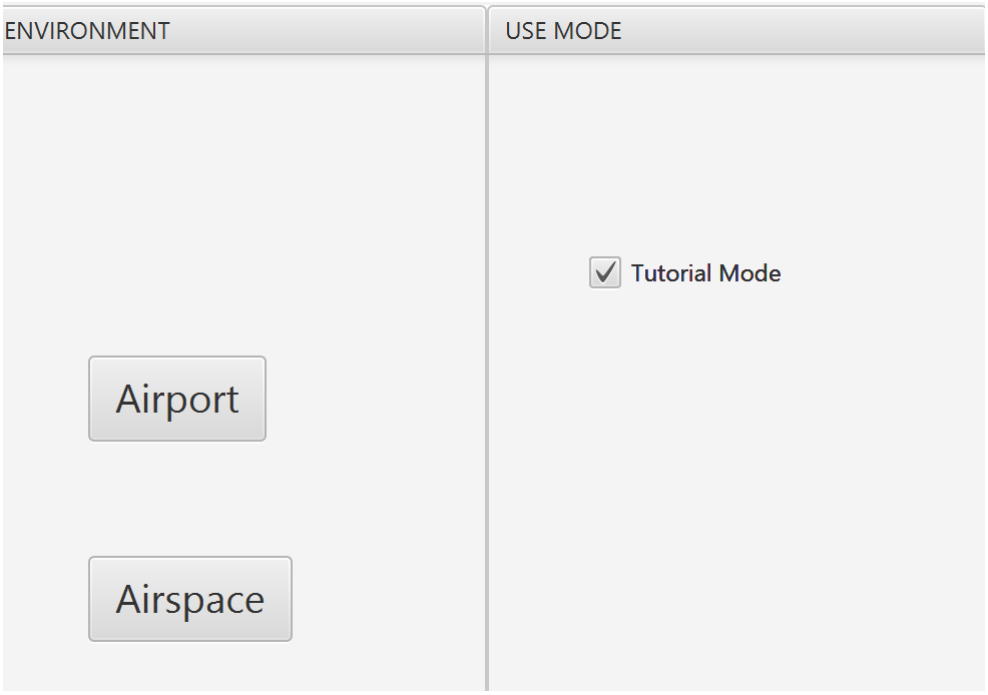


Figure 5.27: Screenshot of menu showing JavaFX CheckBox for tutorial mode selection.

As stipulated in requirement 2a-1, advice is shown to the user in the form of on-screen tooltips. The tooltips do not tell the user about efficient strategies for being a controller (they must learn this through experience), rather it tells them how to use the simulator and control the planes. To show many tooltips, it is sensible to show one at a time with a smooth animation (c.f., showing all at a time which would take up too much space). Therefore, tooltips are shown (**Figure 5.28**) in a cyclic manner with a strobe animation, so the appearance of the next tooltip does not startle the user. Given n tooltips, they are displayed in the order $t_1, t_2, \ldots, t_n, t_1, \ldots$ which is made possible by using the queue data structure. All tooltips are enqueued at the start, and each time a tooltip changes, the old tooltip is re-enqueued while the next tooltip is dequeued and displayed. This functionality is implemented in the code snippet in **Figure 5.29**.

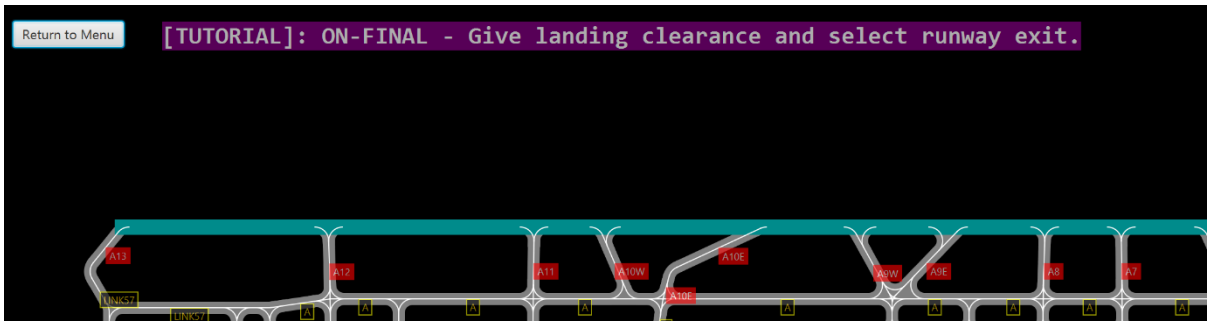


Figure 5.28: Screenshot of airport simulator showing tutorial tooltips.

```java
// dequeue next tooltip, display it and immediately enqueue it
// (analogous for the tooltip color)
public static void changeTooltip() {
    String currentMessage = tooltips.poll();
    tooltips.add(currentMessage);
    tooltip.setText(currentMessage);

    String currentColor = tooltipColor.poll();
    tooltipColor.add(currentColor);
    tooltip.setStyle(currentColor);
}
```

Figure 5.29: Code snippet showing tooltip cycling method changeTooltip() in TutorialManager.java.

Note that in **Figure 5.29**, variables tooltips and tooltipColor are both queues in parallel to cycle the tooltip message and its color. This method is called every 8.5s as determined by the method tick() in **Figure 5.30**, which also varies the opacity between 0 and 0.7.

```java
/**
* Vary opacity. 1s to fade in. 6s no change. 1s to fade out.
* Finally, change the tooltip.
*/
public static void tick(long elapsedMilliseconds) {
    currentMillisecond += elapsedMilliseconds;

    if (currentMillisecond < 1000) {

        tooltip.setOpacity(0.7 * currentMillisecond / (double) 1000);

    } else if (currentMillisecond < 7500) {

    } else if (currentMillisecond < 8500) {

        tooltip.setOpacity(0.7 * (8500 - currentMillisecond) / (double) 1000);

    } else {
        changeTooltip();
        currentMillisecond = 0;
    }
}
```

Figure 5.30: Code snippet of message to strobe the tooltip opacity and change the tooltip every 8.5s (in TutorialManager.java).

5.1.7 Normal Continuous Mode (Requirement 2B)

The implementation of the simulator running normally and continuously is already present through the implementation of requirements 1a through to 1e. This mode of use does nothing special and does not require further development; the simulator can be started as it is.

5.1.8 Realism (NFR-1)

Realism has been achieved to a high standard by many parts of the simulator. This section will discuss some of the implementations to increase realism.

One obvious aspect of realism implemented is the realistic flying characteristics of aircraft. An unrealistic implementation would be to have the aircraft perform on-the-spot turns or magically change their altitude by 5,000ft in an instant. However, our implementation has avoided this by incorporating a rate of change for the altitude, heading, and speed values as discussed in **5.1.5**. Additionally, the maps are scalable to their real-life size as there is a nautical mile per pixel constant specified in all airport and airspace encodings (**Table 5.1** and **Table 5.2**) to ensure planes travel at the correct speed in the simulator (i.e., 250 knots in the simulator is 250 knots in real-life). Both these aspects in the implementation contribute to the simulator being to scale in terms of time and physical size.

Another quality to enhance realism is that only commands which exist in real-world ATC are in the simulator (i.e., there is no magical auto-pilot instruction if the user wants to take a break from the simulator). Instructions such as a go-around, taxi via some path, or final approach clearances all exist in real-life (end of **2.2**).

Furthermore, the detection of ATC rules being violated (**5.1.3**) is based on real-world ATC operations too. Obviously, a pilot would initiate a go-around if the runway was occupied and hence this feature is present in the simulator. Although the minimum separation warning system for the airspace simulator cannot perfectly replicate reality due to the complication and case-specific nature of the regulations, they are still based and inspired by the real-world rules [30].

Finally, the implemented airports and airspaces exist in the real world. The simulator currently includes London Heathrow airport, Birmingham International airport, and the airspace above Tokyo. To ensure realism, the airport and airspace encodings were made with reference to real technical charts of airports [31] [32] as well as satellite imagery [25]. This offers the advantage that the environments the users can train in are representations of real-world environments.

5.1.9 Usability (NFR-2)

Usability is a crucial part of good software design. The coherent interaction between a human and a system relies on good usability, as with poor usability users are more likely to become confused and frustrated through not understanding the functionality behind the interface [33]. Particularly when technology is used in safety-critical workplaces such as ATC, it is paramount that these systems are naturally easy-to-use to reduce the risk of mistakes which could jeopardize safety. This section discussing the quality of the simulator's UI will only make some basic remarks, as this is not a full study on human computer interaction (HCI).

In the airport simulator, we can see the GUI design (**Figure 5.31**). The design features a moderate range of color contributing to an aesthetically pleasing design whilst not becoming confusing for the user, which adheres to one of Nielsen's 10 well-regarded principles for interaction design: the aesthetic and minimalist design principle [34].

Figure 5.31: Screenshot of the airport simulator.

All aircraft which are pending some form of action from the controller have a blue status tag – such as planes with "PUSHBACK" as their status awaiting permission to pushback and planes with "LINED UP" as their status awaiting takeoff clearance. Additionally, planes which are on their way taxiing and are not necessarily awaiting an instruction have a gray status tag. Not only is this a consistent coloring scheme but is an appropriate usage of certain colors as the different colors draws different amounts of attention depending on the situation – in this case blue draws more attention than gray. The color red has also been appropriately used for important visual elements which quickly needs the user's attention, such as when planes lose their required separation as discussed in **5.1.3**.

Furthermore, there are some aspects of the design which aid perception of the simulator. The way in which airplanes are displayed using an airplane icon on top of the visual representation of the airport and background follows the figure ground principle – one of the Gestalt principles [35] which are centered around human visual perception [36]. Although seemingly obvious, this design choice for the UI helps the user intuit seamlessly that there are planes against the background and nothing else. To further support this, the large colored information tags for each aircraft can also be considered to comply with yet another Gestalt principle: the focal point principle [35]. This is because the combination of size and color on these information tags will draw the user's attention to the aircraft – exactly as intended since the planes need to constantly be monitored and controlled. Conversely, the labels for the name of each taxiway are much smaller because this information is less important to be noticed.

Non-compliance with other principles, such as Nielsen's principle of user control and freedom [34], should not necessarily be seen as a weakness of this system. This is due to the nature of this system being a training simulator. Particularly in the case of Nielsen's principle of user control and freedom which aims to let users leave from a mistake, it is arguably better to avoid this principle in the simulator as a user can benefit by learning from their mistakes – for example if a user clears a plane to take-off at the wrong time and causes a catastrophic situation, they can watch what they have done and learn not to repeat it. However, this principle has been adhered to in the main menu with a 'back' button.

5.1.10 Performance (NFR-3)

Midway through development, some occasional slowdowns in the system were noticed, particularly on a heavy map such as London Heathrow which had lots of entities on-screen. This could be due to the system approaching its limits of its memory allocation (my instinct highly doubts this), the operating system running many background processes and taking up processor time, or the graphics card reaching a limit (this is also doubted since the simulator only displays 2D shapes). Regardless, the intermittent slowdowns were unexplained and needed to be rectified if possible as it was hindering the experience of using the simulator.

Previously, the system ran on 'ticks' which repeated ad infinitum, where each 'tick' represented the same duration of elapsed time. However due to the slowdowns, the time interval between 'ticks' stopped being constant and therefore the simulator was not running to proper time. An ingenious trick to prevent this problem was, rather than fixing the issue of non-consistent time 'ticks', to allow this to happen and instead record the difference in time between each 'tick'. This way, the time elapsed between adjacent 'ticks' is taken into account and ensures the simulator still ran to proper time. Examples of functions using this elapsed time variable is shown in **Figure 5.32**, **Figure 5.33**, and **Figure 5.34**,

```java
private long previousTime;

@Override
public void handle(long now) {
    doHandle(now - previousTime);
    previousTime = now;
}

private void doHandle(long elapsedNanoseconds) {

    // increment all timers.
    movePlaneTimerElapsed += elapsedNanoseconds;
    spawnPlaneTimerElapsed += elapsedNanoseconds;
    proximityMatrixTimerElapsed += elapsedNanoseconds;
    emergencyFlashTimerElapsed += elapsedNanoseconds;

    // if enough time has elapsed since the last time planes were moved, commit the duration of time
    // ... since then to the simulation.
    if (movePlaneTimerElapsed >= findPeriodFromFrequency(MOVE_PLANE_FREQ)) {
        double secondsToCommit = movePlaneTimerElapsed / (double) SECOND;
        movePlaneTimerElapsed = 0;
        fleet.moveFleet(secondsToCommit);
        approachManager.tick(secondsToCommit);
        EnvironmentController.incrementEntranceTimers(secondsToCommit);
        EnvironmentController.incrementRunwayTimers(secondsToCommit);
        EnvironmentController.recentlyLockedNodesChecker();
        EnvironmentController.runwayOccupancyChecker();
    }
```

Figure 5.32: Code snippet showing code in class AircraftTimer (in AircraftController.java) relevant to calculating and sending the correct elapsed time to commit to parts of the system each 'tick'.

```java
public void moveFleet(double secondsToCommit) {
    // propagate eapsed time to function to move each plane
    for (int i = 0; i < this.size(); i++) {
        airplanes.get(i).tick(secondsToCommit);
    }

    if (fleetType == AIRBORNE) { airborneSeparation(); }
}
```

Figure 5.33: Code snippet showing function moveFleet() (in class Fleet, in Aircraftcontroller.java), receiving and further propagating the elapsed time to commit to the simulation.

```java
// calculate and update the new position of the aircraft, dep. on how many seconds have elapsed since last 'tick'
private void movePosition(double secondsToCommit) {
    double xPixels = MyMath.knotsToPixelsPerSecond(MyMath.resolveHorizontal(heading.live, secondsToCommit * speed.live));
    double yPixels = MyMath.knotsToPixelsPerSecond(MyMath.resolveVertical(heading.live, secondsToCommit * speed.live));
    position = position.add(xPixels, yPixels);
}
```

Figure 5.34: Code snippet showing function movePosition() in classclass Airplane, in Aircraftcontroller.java), receiving and putting the elapsed time into effect with a plane's position.

In these code snippets, we see how the elapsed time is calculated and sent throughout the system. An example of a function caught in the middle of this propagation process is shown in **Figure 5.33**. Finally, **Figure 5.34** shows an example of a function at the end of this propagation process, where the correct number of seconds elapsed since the last 'tick' is put to effect by means of changing a plane's x and y coordinate.

5.2 Notable Technical Achievements

In this section, instead of focusing on describing how specific features were implemented to meet requirements (**5.1**), we will focus on justifying in-depth why the method of implementation was a strong choice or why it was a significant achievement.

5.2.1 Graph Theoretic Structure of Airports

One major technical challenge to this project was the problem of needing to represent a real-life airport on the simulator. At first this seemed very daunting, but a natural solution was quickly identified.

Airports typically consist of runways, taxiways, and parking gates. Considering that these entities are generally paths passing through many points (e.g., a runway), it was decided to use a graph theoretic implementation to store and represent the airport on the system which offered two powerful advantages (for details on the implementation please refer to **5.1.4**).

Advantage 1 – Generality of simulator. Since the system reads all the nodes and edges from the stored text file before processing and displaying the airport graph, it is in fact possible to represent any arbitrary airport if the text file to represent the airport is made. Even airports which are imaginary can be made and tested using this simulator, and edits to airports are easily done by changing the text file. This is a huge advantage to the system as a product because it is generally purposed, rather than specific to the currently implemented airports of Birmingham International and London Heathrow.

Advantage 2 – Applicability of graph theoretic algorithms. With the graph theoretic representation of the airport while the application is running, it becomes possible to run graph theoretic algorithms on the airport to our advantage. The ability to traverse the graph (the airport) using adjacent edges was not just helpful in its own right, but also to implement the A* search algorithm to find the shortest path between two points, which is used extensively throughout the application (implementation further explained in **5.1.5**).

Finally, it is important to note that the graph pertaining to the airport is stored in a similar way to that of an adjacency list (see **Table 5.3**), which is more space and time efficient than alternatives such as an adjacency matrix. This is because the system stores the two endpoints for each of its m edges (TaxiwaySegment), and for each of its n nodes (NavigationNode) stores lists for its incident edges. Given an intersection in a real-life airport generally is not incident to more than 4 or 5 taxiways (just as with roads for cars), we can assume a fixed size list of edges for each node giving us $O(n)$ space complexity overall. On the contrary, an adjacency matrix takes up $O(n^2)$ space and would take $O(n)$ time to find all incident edges to any node by having to scan through the rows of the matrix.

5.2.2 Modified A* Search for Realistic Maneuvering of Aircraft on Taxiways

One challenge to implementing the A* Search algorithm was that the initial direction of travel must be considered when finding an optimal path. This is illustrated in **Figure 5.35**.

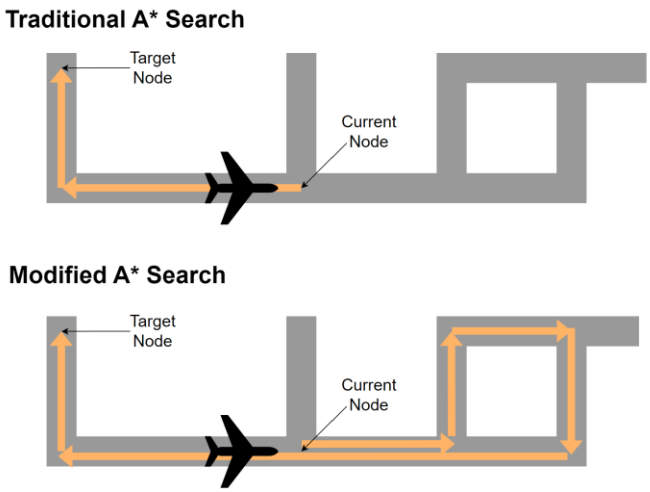


Figure 5.35: Optimal path found using traditional A* Search vs. modified A* Search.

Applying the A* Search to the problem of routing aircraft on the ground unfortunately yields a fundamental inaccuracy where it assumes an aircraft can perform an on-the-spot U-turn at an intersection. In reality, this is not possible; a plane cannot spin on the spot or make extremely sharp U-turns, just as a car cannot do this without enough space around it. Therefore, it is necessary to modify the A* Search algorithm to consider the direction the plane approaches a node, rather than just which nodes are visited. The difference required is illustrated in **Figure 5.35** where the plane ideally should go around some loop of taxiways to turn back on itself. The traditional A* Search algorithm simply considers all adjacent edges, whereas the modified A* Search algorithm only considers edges which are distinct to the previous edge traversed, effectively preventing on-the-spot U-turns. Although this modification may cause a plane to visit a series of nodes twice, this is necessary as it must find a cycle of edges to travel in the backward direction. Ultimately, this innovative trick to modify the A* Search algorithm makes the simulator more realistic.

Finally, to discourage the use of runways for taxiing (runways are best used exclusively for takeoff or landing), the length estimation calculation during A* search adds a very small penalty to using runway edges (**Figure 5.36**). This way, in the case of a tiebreak between multiple paths to a destination, the system should automatically favor the path which uses less runway for the user.

```
// If a runway segment is used for navigation, apply a very small penalty
// (to discourage traversal along the runway when there's otherwise a tiebreaker).
if (predecessor != null) {
    if (nodeEntrySegment.segment instanceof EnvironmentController.RunwaySegment) {
        this.distanceSoFar += penalise(predecessor.distanceSoFar);
    } else {
        this.distanceSoFar += predecessor.distanceSoFar;
    }
}

this.totalEstimate = this.distanceSoFar + this.heuristic;
```

Figure 5.36: Code snippet to show penalization of using a runway segment to route aircraft around the airport (in NodeHeuristic.java).

5.2.3 Queueing of aircraft

An important consideration that is not explicit to the requirement of allowing planes to taxi around the airfield, but which arose after observing the behavior of the implementation is the task of ensuring planes queue behind each other properly. It would be unacceptable for the airport simulator to allow any two planes to be in the same place, or passing through each other, as this would defeat the objective of skillfully maneuvering traffic around the airport. Therefore, an innovative solution was necessary to prevent this unwanted phenomenon.

Since each GroundAirplane object has a coordinate position property and a heading property, the necessary representation for the orientation of an aircraft is already implemented (**5.1.1** and **5.1.2**). Therefore, this only needs to be built upon to implement a queueing system. For a plane at an airport to 'queue' behind another plane and ensure it does not drive into it, it must be checked whether another plane exists in front of each plane.

We can achieve this by using some basic geometry, by checking if a plane exists in a 70-degree cone in front of another plane within a specified distance. If this occurs, the plane will stop. The behavior of this is illustrated in **Figure 5.37** and the implementation is shown in **Figure 5.38**.

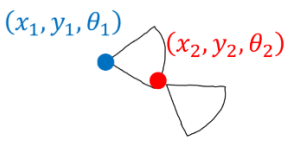


Figure 5.37: A diagram of when a plane may 'detect' another plane in front and queue behind it.

```java
// function will return a speed recommendation with respect to collision avoidance
private Integer collisionAvoidance() {

    // for all aircraft which are nearby to *this* plane, check if the nearby aircraft
    // ... is less than the GROUND_SEPARATION threshold of distnace, within a 70 degree
    // ... cone infront.
    for (Airplane nearbyPlane : proximityList) {

        double nearbyPlaneHeading = MyMath.findHeading(this.position, nearbyPlane.position);
        double absoluteHeadingDifference = MyMath.headingDifference(heading.live, nearbyPlaneHeading);

        // Disregard the nearby plane if it is outside the cone angle
        if (absoluteHeadingDifference > 35) {
            continue;
        }

        double distance = MyMath.distance(this.position, nearbyPlane.position);

        // Disregard the nearby plane if it is outside the cone radius
        if (distance >= GROUND_SEPARATION) {
            continue;
        }

        // Halt the aircraft.
        return 0;
    }

    return null;
}
```

Figure 5.38: Code snippet to show the logic of the queuing system to avoid ground collisions.

The desired behavior is straightforward and obvious and has been implemented successfully as shown in **Figure 5.39**. Although it may appear that the planes have hit each other, this is only a visual effect with the aircraft silhouettes.

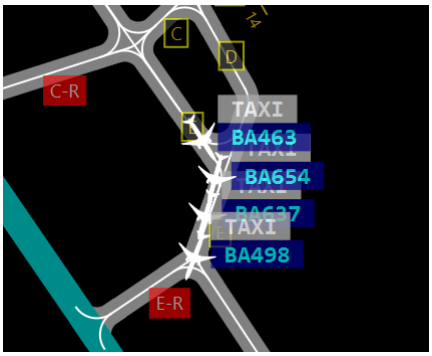


Figure 5.39: Behavior of planes ‘queueing’ behind one another.

5.2.4 Preventing deadlock traffic jams

In **5.2.3**, it is useful to have planes stop before taxiing into another plane on the ground, however it is important to observe that this leads to a new problem emerging within the airport simulator. When two planes are approaching an intersection at a very similar time, sometimes they may both become stuck in a ‘deadlock’, as both planes will not advance due to the queuing behavior. This phenomenon is illustrated in **Figure 5.40**.

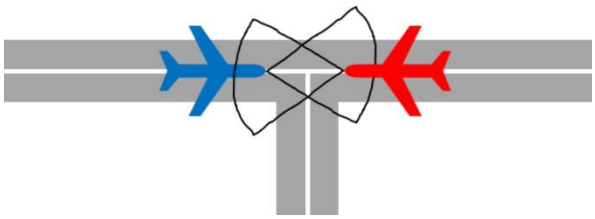


Figure 5.40: Diagram showing planes getting stuck at an intersection.

This situation is clearly unacceptable since this would not normally occur in real life, as usually at an intersection one pilot (or a car, or a person in a more natural context) will move through the intersection before the other does. Therefore, we must take the implementation of **5.2.3** one step further to prevent this issue at intersections.

The solution for this problem was to implement a 'lock' mechanism on all intersections of taxiways or runways. A lock may be acquired by an incident taxiway to an intersection if a plane arrives from that direction – this lock will prohibit planes moving into the intersection from any other taxiway. This way, we can guarantee that planes from only one direction at a time can move through the intersection and therefore we will not get deadlock traffic jams. Analogously, this is identical to the mutual exclusion (mutex) locks which concurrent processes can acquire such that only one process has access to memory to prevent overwrites from other processes [37]. Alternatively, one who is less knowledgeable in computer science may consider this as an imaginary traffic light system where only one set of lights will be green at a time.

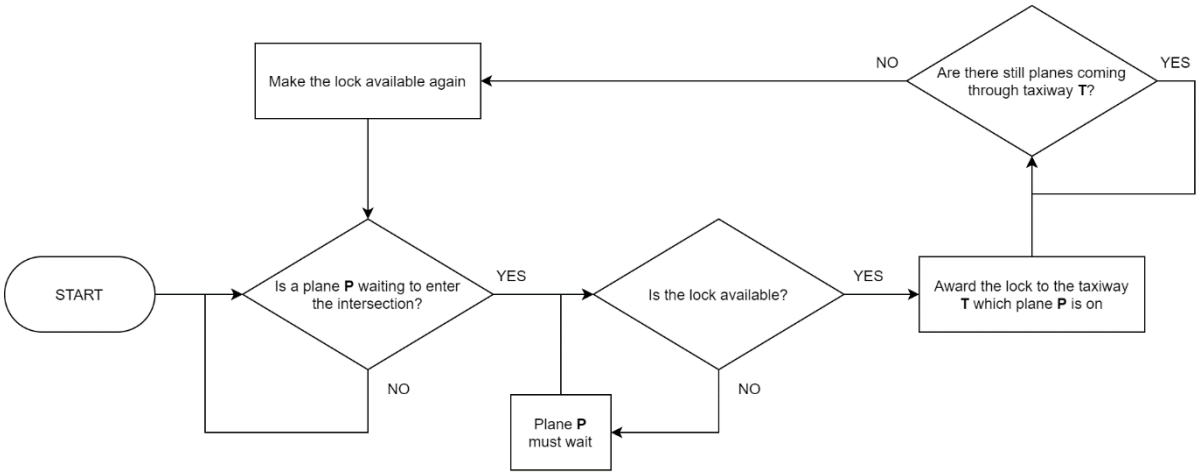


Figure 5.41: Flowchart to show process for when a lock is awarded to a taxiway and released.

5.2.5 Speed Recommendation System

One challenge in this project was to figure out the correct speed for each aircraft, depending on what the aircraft is doing (e.g., taking off). Originally, this was not a straightforward task since it requires

information from many different systems. This whole process needed to work reliably for all planes and sometimes must incorporate a system which may not care what the speed is – for example, the collisionAvoidance() method responsible for stopping or 'queuing' planes (**5.2.3**) should only override the speed of an aircraft if it needs to stop before another plane. Therefore, an appropriate solution would be to have all these systems 'recommend' a speed in priority order to a stack (**Figure 5.42**). The implementation is in movementManager() of class GroundAirplane in AircraftController.java.

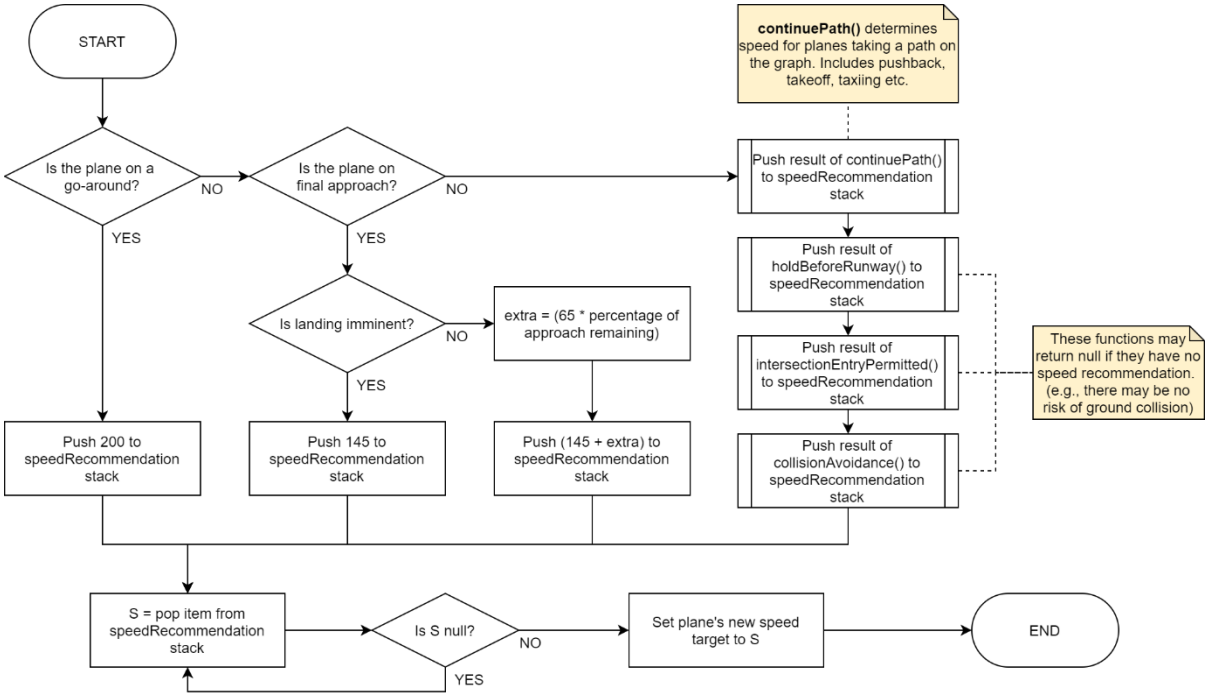


Figure 5.42: Flowchart showing logic of an aircraft's speed decision process (for airport simulator).

Since some systems will not care about the speed and recommend NULL, after all the speed recommendations have been pushed onto the stack, we can simply pop from the stack until we find a non-NULL recommendation. This way, priority is preserved – on the right of the flowchart, the final system to recommend a speed is the collisionAvoidance() method and therefore will be the first speed recommendation checked when values are popped from the stack. The collisionAvoidance() is clearly the most important as no matter what, an aircraft must stop if it sees another plane in front and therefore this method's speed recommendation will override all others, such as holdBeforeRunway().

All routes of the flowchart provide for a non-NULL recommendation (continuePath() always returns an integer), and therefore an aircraft will always have a speed to set as its target. This speed recommendation system is another example of innovative programming to design a versatile solution. The speed recommendation system for the airspace simulator is far simpler and won't be discussed here.

5.2.6 A customized Mathematics library

Java includes a math library with many convenient mathematical functions to use, such as a maximum function or a square root function [38]. However, the raw functions within this provided library may not suffice for all the calculations required in a project. This case applied to our simulator where some involved geometric calculations needed to be performed regularly and by different parts of the system. This warranted the development of a unique class called MyMath (in MyMath.java) to act as a mathematics library for this project. Although this is not necessarily an impressive achievement, the implementation of this class was a strong choice as it improves modularization and maintainability of the code.

For example, two popular methods in MyMath used throughout the simulator are the resolveVertical and resolveHorizontal function (**Figure 5.43**), which are used to resolve the vertical and horizontal component of a scalar with some specified direction. These two methods are very simple and are effectively just one trigonometric operation. However, by making and using these two methods, meaning is put to this trigonometric operation through the method name, and code does not need to be re-used which heavily aided development. The existence of these methods has even facilitated more complex methods made in the MyMath class, such as the rotateVector method (**Figure 5.44**) which rotates a 2-dimensional vector by a specified angle clockwise.

```
protected static double resolveHorizontal(double heading, double quantity) {
    return Math.sin(degreesToRadians(heading)) * quantity;
}

protected static double resolveVertical(double heading, double quantity) {
    return (-1) * Math.cos(degreesToRadians(heading)) * quantity;
}
```

Figure 5.43: Code snippet for resolveHorizontal and resolveVertical methods in MyMath.java.

```
// clockwise rotation by specified angle
protected static Point2D rotateVector(Point2D vector, double degreesToRotate) {

    double originalX = vector.getX();
    double originalY = vector.getY();

    // find the new X and Y contributions from both the rotated X and Y components of the original vector.
    // positive X is in direction 090, and positive Y is in direction 180 - hence adding these to the
    // ... degrees to rotate when resolving the new X and Y contributions.
    double contributionXoriginalX = resolveHorizontal(90 + degreesToRotate, originalX);
    double contributionYoriginalX = resolveVertical(90 + degreesToRotate, originalX);
    double contributionXoriginalY = resolveHorizontal(180 + degreesToRotate, originalY);
    double contributionYoriginalY = resolveVertical(180 + degreesToRotate, originalY);

    double resultantX = contributionXoriginalX + contributionXoriginalY;
    double resultantY = contributionYoriginalX + contributionYoriginalY;

    return new Point2D(resultantX, resultantY);
}
```

Figure 5.44: Code snippet for rotateVector method in MyMath.java.

5.3 Coding Practices

Where possible and not overly challenging, common software engineering principles have been followed. For example, the open-closed principle which is one of the five popularized SOLID principles, states that software entities such as classes or functions should be open for extension but closed for modification [39]. The rationale here is that it is not desirable to change existing functionality to incorporate new features, and that it should be possible to add new features without altering the existing code [40]. From the start it was known that the solution would become complex and therefore following this principle to ensure the solution was future-compatible was a heavy point of focus during development. This was in the interests of facilitating long-term development as well as good design quality. However, no software development is perfect and inevitably there was refactoring of the solution to incorporate changes, since it was never possible to perfectly foresee how all features would be implemented from early on.

Another example of a SOLID principle that has been followed where possible is the single responsibility principle, where classes or functions should have one responsibility and not perform multiple different tasks [41]. Following this principle is advantageous as it simplifies the implementation, as well as improving the understandability and therefore the maintainability of the code. For example, previously, all the code which manages the visual elements for a plane (such as the icon and information tag) was part of the main Airplane class (in AircraftController.java). However, this was changed during development to have the files AircraftVisualElementsAirborne.java and AircraftVisualElementsGround.java be solely responsible for showing the visual elements. Similarly, the AirborneAircraftControls.java and GroundAircraftControls.java files are solely responsible for showing and managing the user controls for airplanes. Furthermore, this single responsibility principle has been followed well especially in the functions written in MyMath.java, which perform some calculations (often making use of other functions) and output one required answer.

As much as various software engineering principles have been followed, one weakness of the software code is that it is messy on a grand scale. There are over 20 java files in the same folder, where some are for more core parts of the system, and others are very small files. Although the act of using separate files is a good choice in itself, this could be improved by putting files in named folders to further structure the code. This was unfortunately not done within this project due to the time constraints prohibiting a full restructuring of the files and ensuring the application still worked afterwards.

Finally, out of consideration for quality software and good performance, sensible development has been conducted through innovative problem-solving and diligent programming throughout the system. That is, there has always been a conscientious effort to achieve the best solution to the problems,

rather than being lazy and implementing a novice algorithm with awful time complexity, such as in **5.1.3** with the minimum separation process running time.

6 Testing

Although some basic verification was conducted after the development of each feature to find bugs and ensure it worked, a full set of tests was only conducted at the end of the project, which is detailed in this chapter.

6.1 Functional Testing

Due to the nature of this project being developing a real-time simulator, it is appropriate to manually inspect the features of the simulator to verify their functionality. Unit testing all the methods and functions may be a popular choice for other projects, however it will not be done here (except for some mathematical functions) as this will be extremely time-consuming and will not actually verify whether the final simulator works (since the simulator is a visual result of all these methods and functions working synchronously together).

6.1.1 Inspection Testing of Simulator Features

Firstly, to ensure the simulator functions as expected, features will be tested uses test scenarios to verify the outcome (**Table 6.1**).

Relevant Requirements	**Test**	**Expected Outcome**	**Outcome as expected?** (if not, detail outcome)
1a-1, 1b-1, 1b-2	Let an aircraft be simulated on screen using the airport simulator.	Aircraft visible somewhere. Aircraft clearly facing a direction.	YES
1a-1, 1a-2, 1b-1, 1b-2, 1b-3, 1b-4, 1b-5	Let an aircraft be simulated on screen using the airspace simulator.	Aircraft visible somewhere. Aircraft clearly facing a direction. Aircraft has an altitude and is shown. Aircraft has a speed and is shown. Aircraft has a displayed destination.	YES
1b-6	Let an airborne aircraft be simulated on screen using the airport simulator	Aircraft visible on mini map.	YES
1c-1	Fly two planes at the same altitude to be within 3 nautical miles laterally, then let them fly past each other.	Initially, a grey line denoting the lateral separation should appear. Then, grey line and label should turn red when distance is less than 3.0 (nautical miles). Finally, they line should return to grey and then disappear as separation is regained.	YES
1c-1	Fly two planes past each other separated by exactly 1,000ft.	No separation line should appear.	YES

1c-2	Let a plane try to land on a runway that has another plane lined up.	Landing aircraft initiates go-around. Message appears for user.	YES
1d-1, 1d-2, 1d-3, 1d-4, 1d-5, 1d-6,	Open the airport simulator on airport "LHR" (London Heathrow).	Airport simulator loads gracefully. All expected taxiways visible. All expected runways visible. All expected parking gates visible.	YES
1d-1, 1d-2, 1d-3, 1d-4, 1d-5, 1d-6,	Open the airport simulator on airport "BHX" (Birmingham International).	Airport simulator loads gracefully. All expected taxiways visible. All expected runways visible. All expected parking gates visible.	YES
1d-8, 1d-9, 1d-10	Open the airspace simulator on airspace "TOKYO".	Airspace simulator loads gracefully. Both airports visible (HND with 4 runways on left, NRT with 2 runways on right) as expected. All expected waypoints visible.	YES
1e-1	Click on a plane which has spawned at a parking gate in the airport simulator.	Control menu shows up offering appropriate options (c.f., inapplicable options such as takeoff clearance for an arriving plane) including PUSHBACK option.	YES
1e-1	Click on a plane which is on a path to cross over a runway in the airport simulator.	Control menu shows up offering appropriate options including CROSS RUNWAY, LINEUP, and TAKEOFF options.	YES
1e-1	Click on a landing plane in the airport simulator.	Control menu shows up offering appropriate options including LANDING CLEARANCE option.	YES
1e-1	Click on a plane in the airspace simulator.	Control menu shows up offering options to increase or decrease attitude, heading and speed, to give approach clearance, to extend path and to clear path.	YES
1e-2, 1d-7	Instruct a plane to extend path to another part of the airport.	Plane should follow the shortest path to the node clicked on.	YES
1e-2, 1d-7	Instruct a plane to stop (clear path).	Plane should not go past the next node it was travelling towards, and stop.	YES
1e-2, 1d-7	Instruct a plane to line-up on the runway	Plane should lineup on the runway and stop.	YES
1e-2, 1d-7	Instruct a plane that is lined up on a runway to takeoff.	Plane should begin takeoff sequence.	YES
1e-2, 1d-7	Instruct a plane that is <u>not</u> lined up on a runway to takeoff.	Plane should lineup on runway and without stopping, begin takeoff sequence.	YES
1e-2, 1d-7	Instruct a plane to cross a runway, where the plane's path goes over a runway multiple times.	Plane should cross over the first encounter of a runway but not any others	YES

1e-2, 1d-7	Give landing clearance to a landing aircraft.	Plane should land on runway then exit the runway.	YES
1e-2	Watch a plane attempt to land which does not have landing clearance.	Plane should initiate a go-around shortly before landing.	YES
1e-2	Instruct a landing plane to go-around	Plane should initiate a go-around immediately.	YES
1e-3	Instruct a plane to change its altitude.	Plane should immediately start ascending or descending to its newly assigned altitude.	YES
1e-3	Instruct a plane to change its heading.	Plane should immediately turn (the closest direction out of CW or anti-CW) to its newly assigned heading.	YES
1e-3	Instruct a plane to change its speed.	Plane should immediately accelerate or decelerate (trail dot separation should change).	YES
1e-3, 1d-11	Instruct a plane to follow a sequence of waypoints (extend path).	Plane should fly towards waypoints (one after another) in selected order. Plane should fly through final waypoint and maintain its target heading.	YES
1e-3	Instruct a plane to clear its path of waypoints.	Plane should maintain its target heading and not follow any waypoints.	YES
1e-3, 1d-11	Give approach clearance to a plane which satisfies the approach criteria (**5.1.5**).	Plane should immediately adjust its altitude, speed, and heading for final approach all the way until it lands. Plane's information tag should go green.	YES
1e-3, 1d-11	Give approach clearance to a plane which satisfies the approach criteria except that it is aiming for the wrong airport. (**Recall**: HND is on left, NRT is on right)	Plane does not enter the final approach mode. Message appears to user indicating a failed approach attempt.	YES
1e-3, 1d-11	Give approach clearance to a plane which satisfies the approach criteria except that it is flying away from the airport.	Plane does not enter the final approach mode. Message appears to user indicating a failed approach attempt.	YES
1e-3, 1d-11	Give approach clearance to a plane which satisfies the approach criteria except that it's altitude is too high.	Plane does not enter the final approach mode. Message appears to user indicating a failed approach attempt.	YES
2a-1	Open simulator on tutorial mode.	Simulator loads gracefully. Tooltips are cyclicly shown to the user on what to do.	YES
2d-1	Open simulator on normal mode.	Simulator loads gracefully.	YES

Table 6.1: Inspection tests of Simulator Features

The fact that all inspection tests passed shows how robust and performant the simulator is. However, this is not a guarantee of flawless software, since there may be faults which arise in very specific and yet undiscovered cases. This is because the inspection tests do not cover a full continuous range of cases and only demonstrate functionality for a typical operation. Although, long-term runs of using the simulator for more than 60 minutes have not yielded any fatal errors.

6.1.2 Unit Testing of Mathematical Functions

To test the correctness and robustness of the MyMath.java mathematical library in the system, discrete unit tests have been made to verify the output of functions. The inputs of these tests can be categorized into some categories:

- **Normal Data**. Inputs which are expected during runtime.
- **Boundary Data**. Input values on edge cases.
- **Extreme Data**. Values of great magnitude near the limit of what can be handled.
- **Erroneous Data**. Inputs with an incorrect data type [42].

Erroneous data will not be tested since such data is never an input; the user can never directly access these functions as they are limited to button clicks. Therefore, testing will be done on only normal, boundary, and extreme data (**Table 6.2**).

Function	Type	Input	Expected Output	Actual Output	Passed?
degreesToRadians	Boundary	0	0.0	0.0	YES
	Normal	180	3.14159…	3.141592653589793	YES
	Normal	360	6.283185…	6.283185307179586	YES
radiansToDegrees	Boundary	0	0.0	0.0	YES
	Normal	Math.PI	180.0	180.0	YES
	Normal	1.5 * Math.PI	270.0	270.0	YES
resolveHorizontal	Normal	Heading: 30 Quantity: 5	2.5	2.4999999999999996	Acceptable
	Extreme	Heading: 90 Quantity: 50000000	50000000.0	5.0E7	YES
	Boundary	Heading: 0 Quantity: 5	0.0	0.0	YES
resolveVertical	Normal	Heading: 0 Quantity: 8	-8.0	-8.0	YES
	Normal	Heading: 150 Quantity: 10	5.0	4.999999999999999	Acceptable
	Normal	Heading: 270 Quantity: 5	-5.0	-5.0	YES
headingCorrecter	Normal	-30	330.0	330.0	YES
	Normal	450	90.0	90.0	YES
	Normal	270	270.0	270.0	YES

	Extreme	360000020	20.0	20.0	YES
findHeading	Normal	Source: (100,100) Dest: (200,200)	135.0	135.0	YES
	Normal	Source: (500,500) Dest: (500,1200)	180.0	180.0	YES
interpolate	Boundary	Source: (500,500) Dest: (500,1200) Ratio: 0	(500.0, 500.0)	(500.0, 500.0)	YES
	Normal	Source: (500,1000) Dest: (500,1200) Ratio: 0.5	(750.0, 850.0)	(750.0, 850.0)	YES
	Normal	Source: (500,500) Dest: (1000,1200) Ratio: 4	(2500.0, 3300.0)	(2500.0, 3300.0)	YES
distance	Extreme	Source: (500,500) Dest: (500,5000000)	4999500.0	4999500.0	YES
	Normal	Source: (500,500) Dest: (1000,1000)	707.10678…	707.1067811865476	YES
	Boundary	Source: (-500,500) Dest: (-500,500)	0.0	0.0	YES
rotateVector	Normal	Point: (-500,500) Rotate: 90	(-500.0, -500.0)	(-500.00000000000006, -499.9999999999999)	Acceptable
	Normal	Point: (6000,0) Rotate: 135	(-4242.640687…, 4242.640687…)	(-4242.640687119285, 4242.640687119286)	YES
	Boundary	Point: (6000,0) Rotate: 360	(6000.0, 0.0)	(6000.0, -1.83697019872103E-12)	Acceptable
headingDifference	Normal	HeadingA: 350 HeadingB: 10	20.0	20.0	YES
	Normal	HeadingA: 10 HeadingB: 350	20.0	20.0	YES
	Normal	HeadingA: 90 HeadingB: 270	180.0	180.0	YES
modArgOffset	Boundary	Point: (500,500) Rotation: 0 Quantity: 0	(500,500)	(500.0, 500.0)	YES
	Normal	Point: (500,500) Rotation: 90 Quantity: 10	(510.0, 500.0)	(510.0, 500.0)	YES
	Normal	Point: (500,500) Rotation: -270 Quantity: 10	(510.0, 500.0)	(510.0, 500.0)	YES
	Normal	Point: (0,0) Rotation: 135 Quantity: Math.sqrt(72)	(-6.0, 6.0)	(6.0, 5.999999999999999)	Acceptable

	Extreme	Point: (500,500) Rotation: 90 Quantity: 0	(500.0, 4000500.0)	(500.00000000244927, 4000500.0)	Acceptable
It is important to note that in a JavaFX scene, the x-axis is positive to the right, and the y-axis is positive to the bottom.					
Testing program can be found in the folder testMaths, in the file TestMaths.java.					

Table 6.2: Discrete unit tests for MyMath functions.

Impressively, out of 35 test cases none have failed or produced a wrong result. There are a few test cases for which the results are labelled "Acceptable" since there is a negligibly small error in the output value. This is likely when real numbers with a decimal component in base-10 cannot be represented exactly with a floating-point representation in binary (i.e., base-2) [43] and is passed through trigonometric functions. These errors are so small in magnitude that they do not affect the simulation in a noticeable or negative way, nonetheless it is possible to rectify this issue via rounding. Overall, this demonstrates a high-level of correctness and robustness in these mathematical functions.

6.2 Remarks on Non-Functional Qualities

It is difficult to test the non-functional qualities, and therefore there will only be a brief discussion on how well these requirements were implemented.

Overall, the simulator gives a good sense of realism. One who has observed aircraft through a tracking application such as Flightradar24 [44], or in-real life can see that the simulator offers a very similar experience. This level of realism achieved can be attributed to the implementation discussed in **5.1.8 Realism (NFR-1)**.

Usability is generally good throughout. Although the purpose of some controls may not be immediately obvious to a novice user due to the nature of this software being a highly technical simulator, the controls are otherwise straightforward and are highly responsive; actions lead to immediate affects in the simulation. Furthermore, as discussed in **5.1.9 Usability (NFR-2)**, the design of the user interface adheres to several common principles.

Finally, the performance of the simulator is generally good. Even after long periods of use, the simulator is responsive and does not slow down. This is an excellent indicator of high-quality software and is worthy of appraisal especially with so many processes within this system constantly ongoing across many Java objects. Part of this high performance can be attributed to sensible programming (i.e., not using novice algorithms with high time complexity), and innovative solutions employed as described in **5.1.10 Performance (NFR-3)**.

7 Evaluation

7.1 System Evaluation

This section will discuss the strengths and weaknesses of the software with reference to the system design and the implementation of requirements.

Overall, the simulator is aesthetic and simple thanks to the JavaFX objects used to represent controls, and airport or airspace entities. The quality of simulation is generally high as airplanes move to-scale and in real-time, offering a prime experience for being an air traffic controller. The simulator has many features which work properly; planes reliably react to all given instructions and the success of these features can be attributed to innovative problem solving (particularly the **Notable Technical Achievements**). Additionally, no fatal errors have occurred during use which shows the robustness of the simulator for long-term use. The proportion of passed tests (as detailed in **6 Testing**) show that functionality works as intended. However, one thing that could be improved is how user-friendly the simulator is; it is conceivable that a user may struggle to understand what to do in the simulator, especially if they are not knowledgeable about aviation.

From an architectural perspective, there could be some improvements. The monolith architecture chosen for this project does not naturally offer structure to the system. This structure must originate from modularity of the code and files. Although the code is relatively modular and reusable with object-oriented functions and classes, the structure on the grand scale is non-existent; all java files are in one folder, and this could be improved (aforementioned in **5.3**). Re-structuring of the files at a high level is paramount should this simulator ever be developed further into a larger product. Fortunately, further development is feasible thanks to the incremental development methodology which allows more features to be added to make newer 'versions' of the product – although care must be taken with this to not break existing functionality.

Finally, a strength of the simulator is that it has been driven by research to enhance realism (**5.1.8**). This includes but is not limited to, designing accurate airports (e.g., London Heathrow) based on real airport charts [31], enforcing real ATC regulations (e.g., minimum separation [30]), and implementing realistic phraseology. This research-driven development contributes to a realistic experience offered by the simulator.

7.1.1 Informal Opinions

Although I likely have the most thorough idea about how well the project has progressed, it is important to consider an alternative perspective and gather other opinions. Although the data collected here will be limited to a few informal opinions, not considering any other perspective can lead to

psychological confirmation bias, where my own knowledge or expectation about the project could influence my opinion [45].

Some close friends were asked to give their general thoughts about the application developed in this project, and anything which stood out to them.

Respondent 1 – "I think that your attention to detail in maintaining realism is very impressive, and that there are clearly a lot of different components that work together seamlessly. And it looks cool."

Respondent 2 – "As someone who has no idea how this works, the control panel can seem a bit daunting. It does look professional though."

It is healthy to see both comments in favor of the developed application, and comments suggesting areas which can be improved upon. Although the control panel could be made to be less daunting, the job of being an air traffic controller is likely daunting to everyone the first time they see what it involves.

These informally gathered opinions should be treated with caution for a few reasons. The respondents were Computer Science students, and none of the respondents were formally introduced to the whole application before giving their opinion. This makes the data unrepresentative, but the opinions were still useful to learn.

Although not the case for this project, if the development was for example contracted in a professional setting, it may be appropriate to conduct a full investigation by trialing the software and seeing how users of varying backgrounds with respect to aviation cope for the first time using the simulator. This could offer a detailed insight into the typical interaction between the user and the system which can inform about the UI quality and design of the simulator.

7.2 Requirements Evaluation

Overall, the final requirements (**2.3**) were complete in the sense that they included many aspects which the simulator needed. There were no huge gaps in the requirements which would have rendered the simulator inoperable. Generally, the development of the project focused on these requirements and did not veer off-course into developing things irrelevant to the requirements. This can be attributed to good thinking and planning of the objectives and requirements at a high level.

However, one thing the final requirements lacked on a lower level was the implementation to issues which were less obvious from the start. For example, where the final requirement 1e-2 (**Table 2.3**) specified that a user must be able to issue and have executed a command to taxi the aircraft around the airport, this failed to take into account the presence of other planes on the airfield which was foreseeable and should've been thought through better. More thought into this aspect originally could have led to a realization of needing a requirement to let planes 'queue' behind one another (**5.2.3**), but

something which may have been hard to foresee was the further emergent phenomenon of planes becoming stuck in a deadlock at an intersection (**5.2.4**) as a result of this queuing behavior. This shows that even if the requirements had been more considerate of potential problems and the necessary solutions to them, there will always be difficulty generating a perfect set of requirements which encompasses a solution to every phenomenon that may occur.

Additionally, some of the requirements could have been more detailed when it comes to how a feature may be implemented. For example, the requirement never detailed that a taxiway's name had to be labelled on-screen, but this was implemented as it was helpful to have. This was not possible due to the limited time available for a lengthy planning process, which meant that innovation had to occur during development as well as during the initial planning stage. Fortunately, this was not detrimental in this project, however in a software project over a longer time span and involving many people, it would be ideal for a more rigorous elicitation of requirements to identify how the implementation should proceed in strong detail.

Finally, the change in requirements from the initial requirements to generate the final requirements is worthy of appraisal. Although the original requirements are impressive in the ideas generated, realizing that there is insufficient time to implement all these ideas can be attributed to forward-thinking and good project management. Therefore, the main outcome of the evolution in requirements of significantly reducing the scope during the project was a pivotal facilitator in the project's success. Furthermore, these changes in requirements also gave rise to the non-functional requirements (**Table 2.5**) to encourage various qualities to be part of the system.

7.3 Project Evaluation

Despite the project undergoing various changes to its plan, overall, it has been successfully carried out. Most of the important requirements have been implemented and there exists a robust and working air traffic control simulator. Although various parts of the scope were removed, the simulator has reached a state on-time where it can be used to train and educate users on their ATC skills.

Looking at the proportion of requirements implemented more carefully, we see that of the final requirements (**2.3**), all except for requirement 2a-2 have been implemented. The one requirement missed was a Should Have requirement and therefore although unfortunate, its omission is not detrimental to the system. Therefore, this is another point of appraisal for the project as the quantity of requirements implemented was far from a minimum attainment. It is also impressive that the majority of Must Have requirements from the original requirements (**2.1**) were implemented in the end since this original vision of the project was changed heavily since the beginning. However as expected, many of the Should Have and Could Have requirements from the original set have been left unimplemented due to the scope reductions.

If this project were repeated, it could be helpful to only plan for part of the project's development at the start, and then periodically extend this plan as the project went on according to progress. Instead of having a full project plan from the start, this project management strategy would be more robust and tolerant against unexpected obstacles. The plan would be extended in a more flexible nature, choosing the best options given the time and resources remaining, and overall be a more versatile and superior strategy to manage this project.

8 Conclusions and Future Work

8.1 Project Outcome

The outcome of this project is a functional and robust air traffic control simulator which incorporates many aspects of realism as well as features which distinguish the product from being a game for children. The educational aspect is present both directly – through the tutorial mode and implementation of visual warnings when for example planes go wrong, and indirectly – through the natural learning and skill acquisition a user will experience through using the simulator for practice.

Overall, the careful design and development that has gone into the system has resulted in a product that is relatively straightforward to use; the only confusion may arise from how to be an air traffic controller, but this is somewhat inevitable for this sort of application. The simulator is generally lag-tolerant (see **5.1.10**) and has attempted to adhere to usability principles where possible (see **5.1.9**). The project has suffered various scope reductions however the iterative development methodology and the planning of these removed features within the original requirements (**2.1**) serves as a foundation for possible project extensions.

The personal outcome of this project is valuable experience in project management and developing a large application. Lessons have been learnt from this experience and particularly where things went in an unexpected way (e.g., longer than anticipated development times). Furthermore, my confidence in programming, especially using Java and building graphical applications has increased significantly since the start of this project.

8.2 Project Extensions

From using an incremental development strategy where the simulator is built piece-by-piece [13], it is possible to continue adding more features to the system. Therefore, the possible extensions to this project we discuss in this section are not just mere ideas but are very possible if the time and resources were spent implementing them.

The possible extensions to the project we discuss will be split into three categories: simulator realism, simulator features, and the application interface.

- **Simulator realism.** Currently the simulator has been made to be as accurate as conveniently possible (see **5.1.8**). However, there is no limit to how realistic or accurate a simulator can become; we can approach reality but never perfectly achieve it in a computer program. As seen in the example of the ATC Pro simulator by Flag Mountain Software LLC and Aerius Designs in **1.1 Existing Systems**, air traffic control simulators can become incredibly

complex and realistic. Some examples of how the simulator in this project could be made more realistic are:

- to incorporate different categories of aircraft and simulate their different flying characteristics instead of modelling all planes homogenously,
- to simulate the delay for pilots to input a control to the aircraft after receiving an instruction from ATC (the user),
- to model airports and airspaces to a better resolution; in the project, some small details in airports and airspaces were not included to avoid complication.

Some of these realism features were envisioned in the original requirements (**2.1**), but were sadly cut out of the project in the final requirements (**2.3**).

- **Simulator features.** There are many additional features which could have been added to this simulator – some were originally planned to be added. One missing feature which is very prevalent when it comes to ATC in real-life is the weather. Adverse weather can have effects including but not limited to pilots being less willing to fly at certain altitudes or speeds, initiation of a go-around during approach, and restriction in available airspace [46]. Simulating the effects of weather would not only be a new feature, but also an extra dimension of realism. Another useful feature would be the ability to save, pause and load the simulator as envisioned in the original requirements (**2.1**). Finally, one feature that is currently unsupported is the simulation regarding runways which cross-over. This was not implemented due to the complexity it would add to the system but it certainly an option when it comes to future development ideas.
- **Application interface.** Although the user interface in the application is acceptable and simple, the aesthetics could be improved by using more responsive and immersive features. This is by no means a necessary extension, but it would improve the quality and feel of the application if there were smooth animations to accompany clicks on the buttons for example. Additionally, the main menu is quite bland, and could be improved into a more colorful and filling design while maintaining its professional look.

One example of future work that does not necessarily extend the simulator software, is the idea of consulting with a professional air traffic controller, or ATC training service to gain further insight and knowledge on the best way to train air traffic controllers. This could be helpful in informing or re-designing aspects of the simulator.

Finally, to take the idea of project extensions one step further to a grander scale, it may be possible to implement larger features which address fundamental disadvantages of the system and to ultimately to expand the scope of the entire application.

- **A graphical airport and airspace builder.** Currently the process of creating airports or airspaces on a text file is quite cumbersome (see **5.1.4**), and such a graphical builder would solve this problem. The user would be able to click and drag entities (nodes, runways, etc.) onto the screen, with the ability to customize them conveniently and precisely. However, such an idea should not be underestimated to be a small addition; it would take significant development and could be on a scale worthy of its own project.
- **AI control / multi-user control.** In reality, airspaces are divided [47] and controllers across different airspaces indirectly work with each other as planes transition from one airspace to another adjacent airspace. However, in this project, the simulator is limited to controlling one single airport or airspace; the simulation of adjacent airspaces is abstracted to spawn planes randomly and periodically as if they flew across the airspace boundary. A way to improve this is to implement an AI or multi-user control to control adjacent airspaces, like reality. However, this would require immense effort; multiple simulators would need to run in parallel and synchronization, along with an API to manage communication between two instances of the simulator across an airspace boundary. There has even been a suggestion to link this ATC simulator to existing flight simulators where users can fly aircraft such that users can work together both as pilots and controllers, but the scale of these ideas is becoming ridiculous.
- **Voice Control.** Controllers and pilots in real life communicate over a radio frequency [11], however this is not the case in the simulator. In-fact the aircraft never sends any messages or requests to the user in the simulator, only the user sending instructions to the aircraft through button clicks. In the future, a new feature could be implemented to detect and interpret the voice of the user just like the Tower!3D Pro ATC simulator by FeelThere (**1.1**). This would require some challenging work on something I have no experience doing and may not even work properly as envisioned.

8.3 Author's Assessment

Developing an ATC simulator, inspired by other ATC games and simulators which I enjoyed throughout my younger years, as well as my general interest in aviation has been a strong source of motivation. This project shows that a large project can be managed and executed, employing innovative problem-solving and diligent software engineering to produce a high-quality and realistic simulator application, and is therefore a proud personal achievement.

Although the simulator is limited to what was implemented during this project, people can still use it for both a fun challenge and to learn how to be a controller.

9 Glossary

API – Application Programming Interface which defines how different parts of an integrated service should communicate with each other.

ATC (Air Traffic Control) – The service of moving aircraft safely and efficiently through the airspace system [48].

Boeing 787-8 – A popular mid-sized airliner manufactured by Boeing.

Flight Level – A measurement of altitude in units of 100ft (e.g., FL250 = 25,000ft). Strictly, it is the altitude above mean sea level when the sea level pressure is 1013.2mb [49].

Go-Around – An aborted landing, resulting in the redirection of the aircraft back for another approach.

GUI (Graphical User Interface) – A form of software interface for users which uses visual components that offer interaction.

Wake Turbulence – The effect of aircraft producing lift, leaving two counter-rotating vortices behind the aircraft [50].

The terms *airplane*, *aircraft*, and *plane* have been used interchangeably through this document.

10 References

Considerable sections of the Project Specification [51], Progress Report [52], and the Presentation [53] for this project have been used and/or adapted in this Final Report without direct citation. Citations originally used in such work have also been included in this Final Report where relevant.

[1] DebbieH, “How long does it take to become an Air Traffic Controller?,” AviationCareerHub, 21 May 2021. [Online]. Available: https://www.aviationjobsearch.com/career-hub/articles/career-advice/general/how-long-does-it-take-to-become-an-air-traffic-controller. [Accessed 10 April 2023].

[2] Flag Mountain Software LLC and Aerius Design, “ATCProSim,” Flag Mountain Software LLC and Aerius Design, 2019. [Online]. Available: https://www.atcprosim.com/. [Accessed 4 April 2023].

[3] FeelThere, “Tower!3D Pro by FeelThere,” [Online]. Available: https://feelthere.com/shop/air-traffic-control/tower3d/tower3d-pro/. [Accessed 23 April 2023].

[4] FeelThere, “Tower!3D Pro on Steam,” Steam, [Online]. Available: https://store.steampowered.com/app/588190/Tower3D_Pro/. [Accessed 23 April 2023].

[5] J. Scott, “Inside Teesside Airport's air traffic control tower,” BBC News, 30 April 2023. [Online]. Available: https://www.bbc.co.uk/news/uk-england-tees-65264092. [Accessed 01 May 2023].

[6] C. Allen, “MoSCoW Method,” ProjectManagement.com, 01 August 2020. [Online]. Available: https://www.projectmanagement.com/wikis/483402/MoSCoW-method. [Accessed 28 April 2023].

[7] Chartered Management Institute, in *Setting SMART Objectives Checklist 231*, pp. 1-2.

[8] M. Ketkar, “True Airspeed Calculator,” Omni Calculator, 23 February 2023. [Online]. Available: https://www.omnicalculator.com/physics/true-airspeed. [Accessed 10 April 2023].

[9] K. Hughes, “74Gear - Pilots Make Terrible Decision in 2:22 | Hollywood vs Reality,” 20 June 2021. [Online]. Available: https://www.youtube.com/watch?v=jGj9X2hHiU4. [Accessed 25 April 2023].

[10] P. Currie, Director, *2:22.* [Film]. United States, Austrailia: Lightstream Pictures, Pandemonium Films, Walk The Walk Entertainment, 2017.

[11] Federal Aviation Authority., “Chapter 4. Section 2. Radio Communications Phraseology and Techniques,” in *Aeronautical Informational Manual*, United States Deparment of Transport & Federal Aviation Authority.

[12] I. Sommerville, “2.1 Software process models,” in *Software Engineering (10th ed.)*, Pearson, 2015, p. 50.

[13] deepsource, “Incremental Development,” [Online]. Available: https://deepsource.com/glossary/incremental-development/. [Accessed 18 April 2023].

[14] tutorialspoint, “MVC Framework - Introduction,” tutorialspoint, [Online]. Available: https://www.tutorialspoint.com/mvc_framework/mvc_framework_introduction. [Accessed 28 April 2023].

[15] aws, “What Is An IDE (Integrated Development Environment)?,” aws, [Online]. Available: https://aws.amazon.com/what-is/ide. [Accessed 16 April 2023].

[16] A. S. Gillis, “What is object-oriented programming?,” TechTarget, July 2021. [Online]. Available: https://www.techtarget.com/searchapparchitecture/definition/object-oriented-programming-OOP. [Accessed 28 April 2023].

[17] Thorben, “OOP Concepts for Beginners: What is Polymorphism,” Stackify, 17 December 2021. [Online]. Available: https://stackify.com/oop-concept-polymorphism/. [Accessed 28 April 2023].

[18] javaTpoint, “Factory Method Pattern,” javaTpoint, [Online]. Available: https://www.javatpoint.com/factory-method-design-pattern. [Accessed 28 April 2023].

[19] J. Archbold, “Structural Design Patterns,” in *CS261 Software Engineering - Implementation*, 2022.

[20] Oracle, *Class Point2D - JavaFX (documentation).*

[21] airportsbase, “Airport codes Tokyo, Japan (JP),” [Online]. Available: https://airportsbase.org/Japan/all/Tokyo. [Accessed 15 April 2023].

[22] A. J. Elliot and M. A. Maier, “Color Psychology: Effects on Perceiving Color on Psychological Functioning in Humans,” *Annual Review of Psychology*, vol. 65, pp. 102,103,109, 2014.

[23] D. N. “10 reasons to go around,” Aircraft Owners and Pilots Association, 5 May 2006. [Online]. Available: https://www.aopa.org/news-and-media/all-news/2006/march/flight-training-magazine/10-reasons-to-go-around. [Accessed 25 April 2023].

[24] V. I. Voloshin, “Chapter 1 Basic Definitions and Concepts,” in *Introduction to Graph and Hypergraph Theory*, New York, Nova Science Publishers, Incorporated, 2009, p. 5.

[25] Google Maps.

[26] M. Arnot, “Up in the Air: Who Determines a Plane's Altitude?,” The Points Guy, 01 September 2018. [Online]. Available: https://thepointsguy.com/guide/up-in-the-air-who-determines-a-planes-altitude/. [Accessed 28 April 2023].

[27] Skybrary, “BOEING 787-8 Dreamliner,” [Online]. Available: https://www.skybrary.aero/aircraft/b788. [Accessed April 28 2023].

[28] D. Poole and A. Mackworth, “3.6.1 A* Search,” in *Artificial Intelligence 2E Foundations of Computational Agents*, Cambridge University Press, 2017.

[29] D. R. and L. G. , “Analysis of Dijkstra's Algorithm and A* Algorithm in Shortest Path Problem,” *Journal of Physics: Conference Series*, vol. 1566, pp. 6-7, 2020.

[30] ICAO, “Part II.- Methods of application employed by Air Traffic Services. Section 5, Chapter 3.- Wake turbulence,” in *Air Traffic Services Planning Manual*, 1984, pp. II-5-3-3.

[31] ICAO, AERODROME CHART - EGLL.

[32] ICAO, AERODROME CHART - EGBB.

[33] T. Issa and P. Isaias, “Usability,” in *Sustainable Design HCI, Usability and Environmental Concerns*, London, Springer Publishing, 2022, p. 4.

[34] J. Nielsen, “10 Usability Heuristics for User Interface Design,” Nielsen Norman Group, 15 November 2020. [Online]. Available: https://www.nngroup.com/articles/ten-usability-heuristics/. [Accessed 26 April 2023].

[35] UserTesting, “7 Gestalt principles of visual perception,” UserTesting, [Online]. Available: https://www.usertesting.com/resources/topics/gestalt-principles. [Accessed 28 April 2023].

[36] J. Archbold, “Topic 7: Human Computer Interaction 2,” in *CS261 Software Engineering*, 2022, pp. 16-17.

[37] W. Stallings, “Semaphores,” in *Operating Systems: Internals and Design Principles*, Pearson Education Limited, 2018, p. 247.

[38] Oracle, Math Class documentation (Java Platform SE 8).

[39] R. C. Martin, “Chapter 8 OCP: The Open-Closed Principle,” in *Clean Architecture: a craftsman's guide to software structure and design*, Pearson Prentice Hall, 2017, p. 70.

[40] I. V. Abba, “SOLID Definition – the SOLID Principles of Object-Oriented Design Explained,” freecodecamp, 26 April 2022. [Online]. Available: https://www.freecodecamp.org/news/solid-principles-single-responsibility-principle-explained. [Accessed 28 April 2023].

[41] baeldung, “Single Responsibility Principle in Java,” Baeldung, 19 September 2022. [Online]. Available: https://www.baeldung.com/java-single-responsibility-principle. [Accessed 28 April 2023].

[42] CGP, “Section Seven - Design, Testing and IDEs,” in *GCSE OCR Computer Science*, Coordination Group Publications Ltd (CGP), 2020, p. 108.

[43] D. Goldberg, “What Every Computer Scientist Should Know About Floating-Point Arithmetic,” *ACM Computing Surveys*, vol. 23, no. 1 (March 1991), pp. 6-7, 1991.

[44] Flightradar24 AB, Flightradar24.

[45] B. J. Casad and J. Luebering, “Confirmation Bias,” Britannica, 31 March 2023. [Online]. Available: https://www.britannica.com/science/confirmation-bias. [Accessed 20 April 2023].

[46] Skybrary, “ATC Operations in Weather Avoidance Scenarios,” [Online]. Available: https://www.skybrary.aero/articles/atc-operations-weather-avoidance-scenarios. [Accessed 27 April 2023].

[47] NATS, “Introduction to Airspace,” [Online]. Available: https://www.nats.aero/ae-home/introduction-to-airspace/. [Accessed 21 April 2023].

[48] Civil Aviation Authority, “How air traffic control works,” Civil Aviation Authority, [Online]. Available: https://www.caa.co.uk/consumers/guide-to-aviation/how-air-traffic-control-works/. [Accessed 28 April 2023].

[49] NATS, “Altitude, Flight Level and Height,” NATS, [Online]. Available: https://www.skybrary.aero/articles/altitude-flight-level-and-height. [Accessed 29 April 2023].

[50] Federal Aviation Authority, “Chapter 7. Section 4. Wake Turbulence,” in *Aeronautal Information Manual*, United States Deparment of Transport & Federal Aviation Authority.

[51] K. Fairman, Educational Software to train Air Traffic Controllers - Project Specification, 2022.

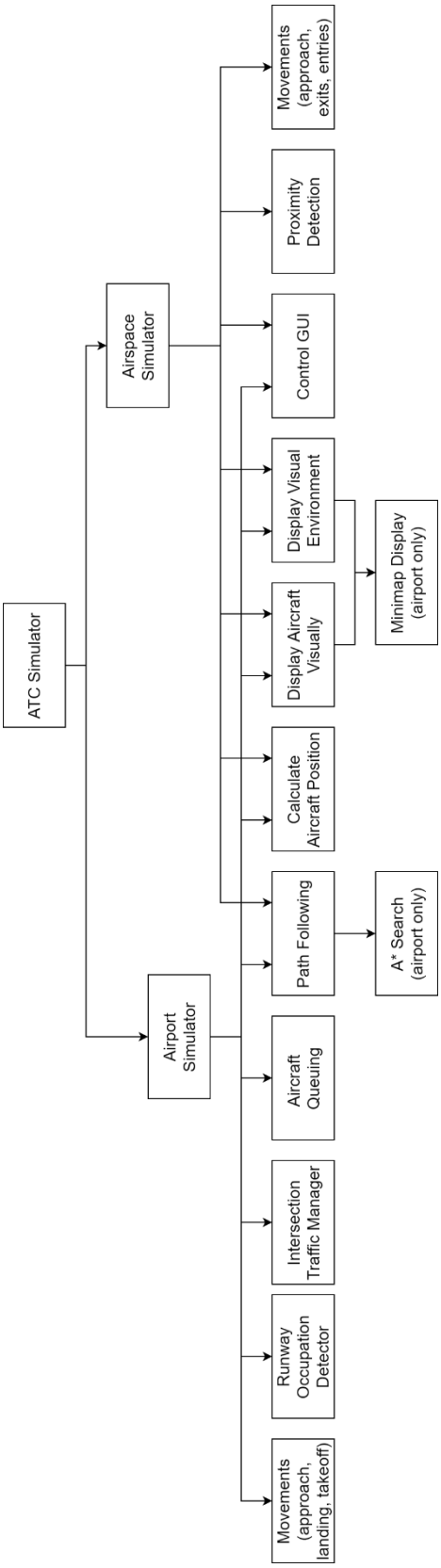
[52] K. Fairman, Educational software to train Air Traffic Controllers - Progress Report, 2022.

[53] K. Fairman, Educational Software to train Air Traffic Controllers - Presentation, 2023.

[54] SKYbrary, “Pilot-Controller Communications - 6.4 Use of nonstandard phraseology,” [Online]. Available: https://www.skybrary.aero/articles/pilot-controller-communications-oghfa-bn. [Accessed 22 April 2023].

11 Appendix

A – Structure of Simulator Components



B – UML Class Diagram (December 2022)

JavaFX Class
Node
Stores
VisualElementsManager
Contains
Stores
Stores
Stores
Stores
Uses to store visual JavaFX Nodes
Is Visually Represented Using
Point2D is javafx.geometry.Point2D
NavigationNode
Gate
RunwaySegment
TaxiwaySegment
is formed of
is formed of
Runway
Taxiway
Uses Taxiways for Exploration
Uses Nodes for Exploration
Java Interface
<<interface>> Comparable
Contains
Contains
Node Heuristic has a predecessor Node and destination Node
Explore
Uses to Analyse Exploration
NodeHeuristic
NodeEntrySegment
is part of
NodeEntrySegment is initialised by Explore
Gate network of the airport
Ground Airplane navigates through EntryNodeSegments
GroundAircraftControls
AirborneAircraftControls
Controls
Controls
AirplaneVariableControl
AltitudeControl
HeadingControl
SpeedControl
Contains
Contains
Contains
<<interface>> Controllable
Controls
Controls
Button is javafx.scene.control.Button
GroundAirplane
AirborneAirplane
Airplane
Controls
Controls
contains
Value
Altitude
Heading
Speed
AircraftValue
AnimationTimer
JavaFX Class
AircraftTimer
AircraftController
SceneController
EnvironmentController
VisualElementsEnum
MyMath
USED BY CLASSES THROUGHOUT SYSTEM
USED BY CLASSES THROUGHOUT SYSTEM

C – Table of Source File Purposes

Source File (in folder src)	**Purpose / Responsibility**
AirborneAircraftControls.java	Manages controls for planes in the airspace simulator. Class extends Controls and implements Controllable.
AircraftController.java	Centrally manages all aircraft within the simulator.
AircraftValue.java	Manages the values (altitude, heading, speed) of an aircraft.
AircraftVisualElementsAirborne.java	Manages visual aspects of planes in the airspace simulator (and for airborne planes in the airport simulator).
AircraftVisualElementsGround.java	Manages visual aspects of planes in the airport simulator.
ApproachManager.java	Manages approaches for aircraft in the airport simulator.
compile.bat	Software compilation.
Controllable.java	Interface for AirborneAircraftControls.java and GroundAircraftControls.java.
Controls.java	Contains superclass for AirborneAircraftControls.java and GroundAircraftControls.java.
EnvironmentController.java	Centrally manages the environment entities and representation within the simulator.
Explore.java	Responsible for graph traversal (e.g., A* Search).
GateManager.java	Manager for tracking parking gate availability in the Airport Simulator.
GroundAircraftControls.java	Manages controls for planes in the airport simulator. Class extends Controls and implements Controllable.
ImageLoader.java	Loads and stores aircraft icon saved locally.
Main.java	Startup file.
MainMenu.fxml	Design of main menu window.
MenuController.java	Centrally manages the main menu functionality.
MessagesManager.java	Manages text messages to be pushed to the user.
MyMath.java	Library for mathematical and geometric functions.
NodeEntrySegment.java	Composite object to aid graph traversal.
NodeHeuristic.hava	Heuristic object to aid A* Search.
SceneController.java	Centrally stores which JavaFX scene to display (e.g., main menu scene).
SimulationEngine.fxml	Design of simulation window.
Status.java	Contains status properties for aircraft.
TutorialManager.java	Manages the appearance of tutorial tooltips on-screen.